Faculty of Informatics Engineering Department of Systems Security and Computer Networking

RAG Based Security Auditor

Prepared by:

Nagham Ashqar

Moustafa Seifo

First release

2026

Abstract :

This project presents a Retrieval-Augmented Generation (RAG)-based security auditing framework for automated, function-level source-code vulnerability detection and classification. The implemented system accepts a user-supplied source-code directory and uses Tree-sitter to extract functions and methods from Python, JavaScript, TypeScript, and TSX files while preserving their original file paths and line locations. Each extracted unit is subsequently analysed by an LLM-based reasoning stage that produces a structured security assessment, including a vulnerable, not vulnerable, or uncertain verdict, one or more predicted Common Weakness Enumeration (CWE) categories, affected source lines, technical explanations, and remediation guidance. The project investigates retrieval augmentation primarily as a mechanism for grounding LLM security decisions in external vulnerability evidence and improving the practical reliability of generated findings, particularly through the control of false-positive reports.

The external knowledge base comprises 764 prepared security records derived from SecureCode v2.0 and contains both vulnerable and secure source-code examples together with associated security metadata. Two alternative retrieval strategies were implemented and evaluated. The first employs microsoft/unixcoder-base to embed the raw source code of both the analysed function and the vulnerable knowledge-base examples, thereby performing code-oriented nearest-neighbor retrieval. The second employs BAAI/bge-m3 to embed natural-language functional descriptions generated from source code, enabling retrieval according to higher-level functional semantics rather than direct code representation. In both configurations, the three highest-ranked knowledge records are retrieved from a persistent ChromaDB collection and supplied, together with the original target function, to GPT-5.1 for vulnerability reasoning. This design enables a direct investigation of how the representation used for retrieval affects the downstream security decision while retaining a common knowledge source and reasoning stage.

The two RAG configurations were evaluated on all 1,230 cases of OWASP BenchmarkPython v0.1, comprising 452 vulnerable and 778 non-vulnerable functions, and were compared with Bandit as a conventional static-analysis baseline. The UniXcoder-based configuration achieved 68.72% precision, 91.37% recall, a 78.44% F1-score, 81.54% accuracy, 75.84% specificity, and a Matthews Correlation Coefficient of 64.82%. The BGE-M3 configuration achieved 61.09% precision, 91.37% recall, and a 73.23% F1-score. Bandit achieved 63.64% precision, 49.56% recall, and a 55.72% F1-score. These results show that the two retrieval representations influence the effectiveness of the same downstream reasoning process differently. Although both RAG configurations achieved high vulnerability recall, the code-oriented UniXcoder representation produced the strongest overall balance between vulnerability detection and false-positive control, whereas the functional-semantic BGE-M3 representation generated a larger proportion of false-positive findings.

The findings demonstrate that retrieval augmentation should not be treated as an inherently beneficial component independent of retrieval design. Rather, the usefulness of RAG for

vulnerability auditing depends on whether the retrieved representation exposes security-relevant similarities that help the reasoning model distinguish genuinely vulnerable implementations from safe but functionally related code. Within the evaluated configuration, direct source-code retrieval using UniXcoder provided the most effective RAG strategy and substantially outperformed Bandit in vulnerability recall and overall F1-score. The resulting prototype therefore demonstrates a practical directory-oriented security-auditing workflow in which structural source-code extraction, external vulnerability knowledge, representation-specific retrieval, LLM-based reasoning, and developer-oriented reporting are integrated within a unified system.

Keywords: Retrieval-Augmented Generation, Software Vulnerability Detection, Source-Code Embeddings, UniXcoder, BGE-M3, Large Language Models, Static Analysis.

الملخص :

يقدّم هذا المشروع إطارًا للتدقيق الأمني قائمًا على التوليد المعزز بالاسترجاع - (Retrieval-Augmented Generation (RAG من أجل الكشف الآلي عن ثغرات الشيفرة المصدرية وتصنيفها على مستوى الدوال. يستقبل النظام المنفذ مجلد مشروع يحدده المستخدم، ويستخدم Tree-sitter لاستخراج الدوال والأساليب من ملفات Python و JavaScript و TypeScript و TSXمع الحفاظ على مسارات الملفات ومواقع الأسطر الأصلية. بعد ذلك، تُحلَّل كل وحدة مستخرجة بواسطة مرحلة استدلال تعتمد على نموذج لغوي كبير، وتنتج تقييمًا أمنيًا منظمًا يتضمن حكمًا من ثلاث فئات :ضعيف أمنيًا، غير ضعيف أمنيًا، أو غير مؤكد، بالإضافة إلى فئة أو أكثر من تصنيفات Common Weakness Enumeration (CWE) المتوقعة، والأسطر المتأثرة في الشيفرة، والتفسيرات التقنية، وإرشادات المعالجة. ويبحث المشروع في استخدام التعزيز بالاسترجاع أساسًا بوصفه آلية لربط قرارات النموذج اللغوي بأدلة خارجية مرتبطة بالثغرات، وتحسين الموثوقية العملية للنتائج الأمنية الناتجة، ولا سيما من خلال التحكم في التقارير الإيجابية الكاذبة.

تتكون قاعدة المعرفة الخارجية من 764سجلًا أمنيًا مُعدًّا ومشتقًا من SecureCode v2.0 ، وتحتوي على أمثلة لشيفرات ضعيفة أمنيًا وأخرى آمنة، بالإضافة إلى البيانات الوصفية الأمنية المرتبطة بها. تم تنفيذ وتقييم استراتيجيتين بديلتين للاسترجاع. تستخدم الاستراتيجية الأولى النموذج microsoft/unixcoder-base لتضمين الشيفرة المصدرية الخام لكل من الدالة الجاري تحليلها وأمثلة الشيفرات الضعيفة أمنيًا في قاعدة المعرفة، وبذلك تنفذ استرجاعًا موجَّهًا بالشيفرة المصدرية بالاعتماد على أقرب الجيران. أما الاستراتيجية الثانية فتستخدم BAAI/bge-m3لتضمين أوصاف وظيفية باللغة الطبيعية يتم توليدها من الشيفرة المصدرية، مما يتيح الاسترجاع وفق الدلالات الوظيفية عالية المستوى بدلًا من التمثيل المباشر للشيفرة. في كلا الإعدادين، يتم استرجاع السجلات الأمنية الثلاثة الأعلى ترتيبًا من مجموعة دائمة في ChromaDB، ثم تُزوَّد هذه السجلات، إلى جانب الدالة الأصلية المستهدفة، إلى GPT-5.1لإجراء الاستدلال الأمني. يسمح هذا التصميم بدراسة مباشرة لكيفية تأثير التمثيل المستخدم في عملية الاسترجاع على القرار الأمني اللاحق، مع الحفاظ على مصدر معرفة ومرحلة استدلال مشتركين بين الإعدادين.

تم تقييم إعدادَي RAG على جميع الحالات البالغ عددها 1,230حالة في OWASP BenchmarkPython v0.1، والتي تتضمن 452حالة ضعيفة أمنيًا و778 حالة غير ضعيفة أمنيًا، كما تمت مقارنتهما مع أداة Banditبوصفها خط أساس تقليدي للتحليل الساكن. حقق إعداد UniXcoder دقة إيجابية (Precision) بلغت 68.72%، واسترجاعًا (Recall) بنسبة 91.37%، ودرجة F1بلغت 78.44%، ودقة كلية (Accuracy) بلغت 81.54%، وخصوصية (Specificity) بلغت 75.84%، ومعامل ارتباط Matthews (MCC) بلغ .64.82%أما إعداد BGE-M3فحقق دقة إيجابية بلغت 61.09%، واسترجاعًا بنسبة 91.37%، ودرجة F1بلغت .73.23%في المقابل، حققت أداة Banditدقة إيجابية بلغت 63.64%، واسترجاعًا بنسبة 49.56%، ودرجة F1بلغت .55.72%وتوضح هذه النتائج أن تمثيلي الاسترجاع يؤثران بصورة مختلفة في فعالية عملية الاستدلال اللاحقة نفسها. فعلى الرغم من أن كلا إعدادَي RAG حققا قدرة مرتفعة على استرجاع الثغرات، فإن التمثيل الموجَّه بالشيفرة باستخدام UniXcoder حقق أفضل توازن إجمالي بين اكتشاف الثغرات والتحكم في الإيجابيات الكاذبة، بينما أدى التمثيل الدلالي الوظيفي باستخدام BGE-M3 إلى نسبة أكبر من التقارير الإيجابية الكاذبة.

توضح النتائج أن التعزيز بالاسترجاع لا ينبغي اعتباره مكوّنًا مفيدًا بطبيعته بصرف النظر عن تصميم آلية الاسترجاع. بل تعتمد فاعلية RAG في تدقيق الثغرات على مدى قدرة التمثيل المسترجع على إبراز أوجه التشابه المرتبطة أمنيًا، بما يساعد نموذج الاستدلال على التمييز بين التطبيقات الضعيفة أمنيًا فعلًا والتطبيقات الآمنة ولكن المتشابهة وظيفيًا. وضمن الإعدادات التي تم تقييمها، قدم الاسترجاع المباشر للشيفرة المصدرية باستخدام UniXcoder الاستراتيجية الأكثر فاعلية ضمن إعداداتRAG ، كما تفوق بشكل ملحوظ على Bandit في استرجاع الثغرات وفي درجة F1 الكلية. وبذلك يوضح النموذج الأولي الناتج سير عمل عمليًا للتدقيق الأمني على مستوى المجلدات، يدمج بين الاستخراج الهيكلي للشيفرة المصدرية، والمعرفة الخارجية المتعلقة بالثغرات، والاسترجاع القائم على نوع التمثيل، والاستدلال باستخدام النماذج اللغوية الكبيرة، وإعداد التقارير الموجهة للمطورين ضمن نظام موحد.

الكلمات المفتاحية :التوليد المعزز بالاسترجاع(RAG) ، اكتشاف ثغرات البرمجيات، تضمينات الشيفرة المصدرية، النماذج اللغوية الكبيرة، التحليل الساكن.

Contents

Chapter 1: Theoretical Background

1.1 Introduction

Software vulnerabilities remain a major source of security risk, arising from weaknesses in program design, implementation, configuration, or use. The Common Weakness Enumeration (CWE) provides a standardized vocabulary for describing such weakness types and supports consistent reporting and comparison across security tools and research studies [1]. This is particularly important for automated vulnerability-detection systems, which must convert model-generated observations into recognized and interpretable security categories.

Automated source-code vulnerability detection has therefore received considerable research attention. Deep-learning approaches have shown that source code can be represented and classified computationally, but their effectiveness is strongly influenced by dataset quality, duplication, class imbalance, and evaluation realism. Chakraborty et al. demonstrated that vulnerability-detection models may rely on superficial artefacts rather than the underlying causes of security weaknesses, leading to reduced performance under more realistic conditions [2]. These findings highlight the need for approaches that go beyond surface-level similarity and provide stronger evidence for security decisions.

Large Language Models (LLMs) have recently introduced additional capabilities for software security analysis because they can interpret source code, follow structured instructions, and generate human-readable explanations. Previous studies have demonstrated the feasibility of applying GPT-based models to vulnerability detection and source-code inspection [3], [4]. However, LLM-based analysis remains susceptible to inconsistent classification, unsupported reasoning, and false-positive findings. In practical auditing, false positives are particularly important because excessive incorrect warnings increase review effort and may reduce confidence in the tool.

Retrieval-Augmented Generation (RAG) offers a mechanism for grounding LLM reasoning in external security knowledge. Instead of relying solely on information encoded in the model parameters, RAG retrieves relevant records from an external knowledge source and incorporates them into the reasoning context. Vulnerability-oriented RAG systems such as Vul-RAG have shown that retrieved security knowledge can support more informed vulnerability assessment by providing examples of vulnerability behavior, causes, and remediation strategies [5].

The effectiveness of RAG, however, depends strongly on how the analyzed source code and the knowledge-base records are represented for retrieval. Functional-semantic retrieval can abstract away language-specific syntax by representing code through natural-language descriptions, whereas code-oriented retrieval preserves implementation-level information that may be security-critical. BGE-M3 supports dense semantic retrieval over textual descriptions [6], while UniXcoder is specifically designed for code representation and code-related retrieval tasks [7]. Comparing these two representations therefore provides a useful way to study how retrieval design influences vulnerability detection and false-positive control.

The proposed system investigates this question through two RAG configurations that share the same knowledge base and downstream reasoning model. The first uses BGE-M3 to retrieve security records through generated functional descriptions, while the second uses UniXcoder to retrieve records through direct source-code representations. In both cases, the retrieved records are treated as supporting evidence rather than as final vulnerability labels.

To support practical source-code auditing, the system also performs directory-level preprocessing using Tree-sitter. The current implementation supports Python, JavaScript, TypeScript, and TSX and extracts functions and methods while preserving their file paths and line locations [8]. Each extracted unit is then subjected to retrieval and LLM-based reasoning, producing a structured verdict of vulnerable, not vulnerable, or uncertain, together with CWE classification, affected lines, technical explanation, and remediation guidance.

The resulting framework combines structural source-code extraction, alternative retrieval representations, external vulnerability knowledge, LLM-based reasoning, and developer-oriented report generation within a unified auditing workflow. Rather than assuming that retrieval augmentation is universally beneficial, this work focuses on how the choice of retrieval representation affects the balance between vulnerability detection and false-positive control.

1.2 Large Language Models

A Large Language Model, or LLM, may be understood operationally as a parameterized neural model trained to estimate probability distributions over sequences of tokens. Depending on the training data, these tokens may represent natural-language words or sub words, programming-language keywords, identifiers, operators, punctuation, and other syntactic elements. Most contemporary LLMs derive from the Transformer architecture introduced by Vaswani et al. [9].

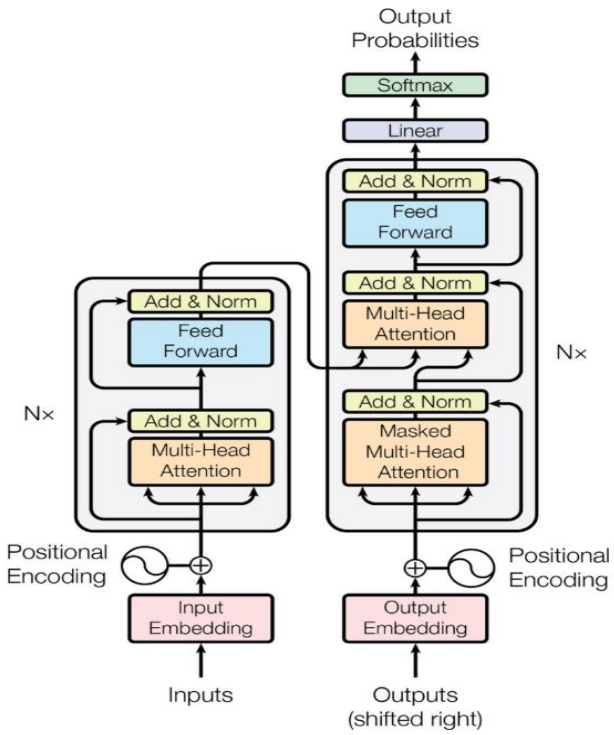

Figure 1 - Transformer's Architecture *[9].*

Figure 1 shows the original encoder–decoder Transformer architecture, reproduced from Vaswani et al. [9]. The encoder is shown on the left and the autoregressive decoder on the right; both are constructed from repeated attention, feed-forward, residual-connection, normalization, embedding, and positional-encoding components.

1.2.1 The use of LLMs for code-specific tasks

When source code is represented as a token sequence, an LLM can learn associations among programming constructs, identifiers, comments, documentation, and natural-language instructions. Chen et al. [10], in their evaluation of Codex, demonstrated that a GPT-family model trained on source code could generate executable programs from natural-language specifications, while also identifying limitations on tasks involving long chains of operations and precise variable–operation associations. This evidence does not establish that an LLM formally understands program semantics; rather, it shows that large-scale language modelling can acquire representations useful for relating textual intentions to code structures. Such a relationship also supports code-to-text operations, including the generation of concise functional descriptions, explanations, and documentation.

Security-oriented studies extend these capabilities from general programming tasks to source-code inspection. Szabó and Bilicki [3] investigated GPT-based static inspection of web-application source code and showed that prompting and preprocessing could be combined to identify and represent security-relevant code. Zhou, Zhang, and Lo [4] subsequently evaluated general-purpose LLMs for software-vulnerability detection under different prompting configurations, reporting

promising but model- and prompt-dependent results. These studies support the use of LLMs as analytical components capable of reading code, describing behavior, identifying suspicious constructs, and generating human-readable findings. They do not, however, establish LLM analysis as equivalent to compilation, execution, symbolic reasoning, or formal verification.

However, the reliability of generated security assessments remains a central concern. Hallucination refers broadly to generated content that diverges from the supplied input, contradicts the preceding context, or conflicts with established knowledge [11]. In code auditing, hallucination may appear as an invented data flow, a nonexistent sanitization step, an unsupported CWE classification, an incorrect vulnerable-line range, or remediation advice that does not follow from the implementation. Linguistic fluency is therefore not sufficient evidence that a finding is technically valid, a further limitation is that models may depend on superficial textual regularities rather than security-relevant semantics. Du et al. [5] evaluated vulnerability-detection techniques using pairs of vulnerable functions and highly similar patched functions. Their findings showed that models had substantial difficulty distinguishing the two, despite the security significance of the comparatively small changes separating them. Vul-RAG consequently represents vulnerabilities through functional semantics, vulnerability causes, and fixing solutions, retrieves relevant knowledge, and then asks an LLM to determine whether the corresponding cause and absence of a fix are present in the target code. This separation between retrieval and assessment is important: similarity identifies potentially relevant evidence, whereas the final security verdict requires examination of whether the target implementation actually exhibits the retrieved vulnerability conditions.

Therefore, within the implemented system, GPT-5.1 serves as the principal vulnerability-reasoning model, while the retrieval stage may provide evidence through one of two alternative representations. In the BGE-M3 configuration, GPT-5-mini first generates a concise functional description of the analyzed function, which is subsequently embedded for semantic retrieval. In the UniXcoder configuration, the source code itself is embedded directly and compared with code representations stored in the knowledge base. In both configurations, the retrieved records are supplied as supporting evidence to GPT-5.1, which performs the final security assessment. This separation allows the project to examine how retrieval representation influences downstream vulnerability reasoning.

1.3 Retrieval Augmented Generation

Retrieval-Augmented Generation, commonly abbreviated as RAG, is an architecture in which a generative model is augmented with information retrieved from an external knowledge source at inference time. Lewis et al. introduced RAG as a combination of parametric memory, represented by knowledge encoded in the parameters of a pretrained sequence-to-sequence model, and non-parametric memory, represented by an explicit dense vector index accessed through a neural retriever [12]. The input is first used to retrieve a restricted set of relevant records, and the generator then conditions its output on both the original input and the retrieved information.

The principal motivation for this architecture is that knowledge stored exclusively in model parameters is difficult to inspect, revise, or extend. Lewis et al. identify knowledge updating and provenance as important limitations of parameter-only models: when knowledge is stored externally, the indexed collection can be modified independently of the generator, and the records selected for a particular query can be inspected as evidence associated with the generated result [12]. RAG therefore provides an architectural mechanism for incorporating task-specific information without requiring all relevant knowledge to be permanently encoded through model training. It should not, however, be described as guaranteeing factual correctness or eliminating hallucinations; its effectiveness depends on whether the retriever selects relevant, accurate, and sufficiently complete evidence.

The theoretical RAG process may therefore be represented in four general operations:

1- Query construction
2- Retrieval
3- Context augmentation
4- Generation or judgement

Query construction determines which representation of the input is used for search. Retrieval ranks knowledge-base records according to their relevance to that representation. Context augmentation combines the selected records with the original input. The generative or reasoning model then produces the task-specific output. The quality of the complete pipeline depends not only on the language model, but also on the representation of the query, the quality and coverage of the indexed knowledge, the similarity function, and the number and relevance of records supplied to the model.

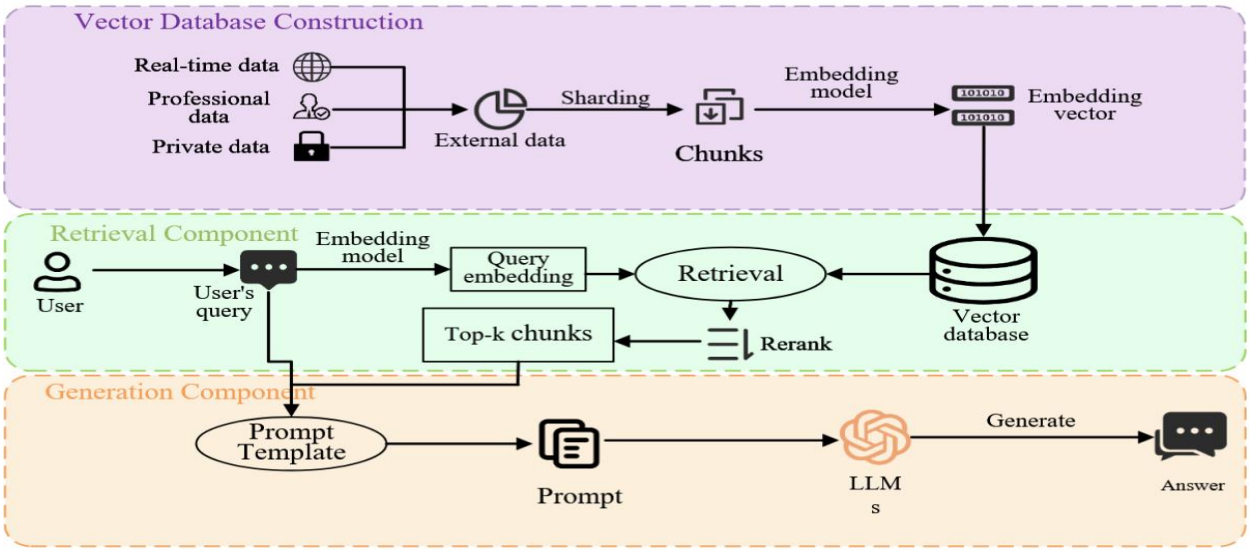

Figure 2 - RAG Technical Workflow *[13]*.

Figure 2 demonstrates RAG technical workflow: i) Vector database construction, which involves calculating semantic vectors for data chunks via data chunking and embedding models to establish the vector database ; ii) Retriever, responsible for retrieving the top-k data chunks most relevant to the user query from the database ; iii) Generator, responsible for integrating the top-k data chunks with the user query and submitting them to the large language model for response generation [13].

1.3.1 Dense semantic retrieval

Dense retrieval represents a query, and each candidate record as numerical vectors in a continuous embedding space. Karpukhin et al. formalised this approach through Dense Passage Retrieval, or DPR, using one encoder for questions and another for candidate passages [14]. Candidate passage representations can be computed and indexed offline, whereas the query representation is computed at runtime. The retrieval system then identifies the k indexed vectors that are closest to, or most similar to, the query vector.

Cosine similarity is a commonly used comparison function for dense textual embeddings. For non-zero vectors q and d, it is defined as:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\| \mathbf{q} \|_2 \| \mathbf{d} \|_2}.$$

The numerator measures the vectors' dot product, while the denominator normalizes their magnitudes. Consequently, cosine similarity primarily measures angular alignment rather than absolute vector length. When the vectors are L_2-normalised, cosine similarity and the dot product are equivalent. Karpukhin et al. discuss inner product, cosine similarity, and Euclidean distance as decomposable alternatives, although their principal DPR implementation uses inner product [14].

Chroma represents cosine comparison as a distance:

$$d_{\cos(\mathbf{q},\mathbf{d})} = 1 - \frac{\mathbf{q} \cdot \mathbf{d}}{\| \mathbf{q} \|_2 \| \mathbf{d} \|_2}.$$

Under this convention, smaller distance values indicate closer vector representations. Chroma also supports squared Euclidean distance and inner-product distance. Importantly, the current Chroma documentation states that the default collection distance is squared L_2, while cosine must be selected explicitly when configuring the collection [15]. Therefore, a report should not state that Chroma returns cosine similarity unless the collection has actually been configured with the cosine space.

Dense retrieval is semantic in the sense that it can rank records as related even when they do not contain exactly the same words. DPR found that dense representations could capture certain lexical variations and semantic relationships that were not recovered through exact term matching [14]. Nevertheless, semantic retrieval remains dependent on the representation learned by the embedding model. A record may be functionally related but security-irrelevant, or security-

relevant but represented too differently to appear among the nearest neighbors. Dense retrieval therefore produces a ranked candidate set rather than an infallible identification of the correct evidence.

1.3.2 Retrieval Representations for Vulnerability Analysis

The representation used for retrieval is a critical design choice in source-code vulnerability analysis because different representations preserve different aspects of program behavior. Raw source code retains implementation-level details such as API calls, operators, control structures, and security-relevant checks, but it is also influenced by syntax, identifier naming, formatting, and programming language. Conversely, higher-level semantic representations can reduce these surface differences and emphasize what a function is intended to do, although they may omit low-level details that determine whether the implementation is secure.

Functional-semantic retrieval addresses this issue by converting source code into a natural-language description of its behavior before retrieval. Vul-RAG provides an example of this strategy by representing vulnerability knowledge through functional semantics, vulnerability causes, and fixing solutions, and by retrieving knowledge with related functionality before the final vulnerability assessment [5]. Such representations can support retrieval across implementations that perform similar operations even when their source-code syntax differs.

However, functional abstraction may remove information that is decisive for vulnerability detection. For example, two functions may both be described as executing a database query while differing in whether they use parameterized queries, direct string concatenation, or input validation. For this reason, functional descriptions are useful as retrieval representations but cannot replace inspection of the original source code.

An alternative is code-oriented retrieval, in which the source code itself is embedded directly using a model pretrained for programming-language representation. UniXcoder supports code representation and code-search tasks and can therefore be used to compare an analysed function directly with code examples stored in a vulnerability knowledge base [7]. This approach preserves more implementation-specific information, although it may be more sensitive to syntactic and language-specific variation than functional-semantic retrieval.

These two representations therefore provide complementary retrieval perspectives. Functional-semantic retrieval emphasizes behavioral similarity, whereas code-oriented retrieval preserves more of the implementation details that may distinguish secure and vulnerable code. The present work evaluates both approaches to examine how retrieval representation affects the quality of the evidence provided to the downstream vulnerability-reasoning stage.

1.4 LLMs for Cybersecurity

Large language models have emerged as tools for cybersecurity automation, with recent surveys cataloging applications across vulnerability detection, malware analysis, threat intelligence, and incident response.

1.4.1 Limitations of LLM Only Approaches

Vul-RAG, a pioneering framework proposed by Du et al. [5]. was born from the observation that LLMs perform poorly (accuracy of only 0.06-0.14) at distinguishing between vulnerable code and its similar-but-patched counterpart, indicating a struggle to capture root causes.

LLMs, when used in isolation for vulnerability detection, exhibit significant limitations. They are prone to generating code with vulnerabilities and struggle to effectively generate security functions that precisely satisfy security requirements, an issue that likely stems from insufficient scrutiny of training data, lack of task-specific knowledge, and inadequate fine-tuning [16].

Moreover, most LLMs are trained on general-purpose corpora and thus lack the domain-specific knowledge essential for effective software vulnerability assessment, tending to rely on shallow pattern matching instead of deep contextual “reasoning” [17]. The comparative study by Saju et al. [18] .confirms this, showing a baseline LLM achieving an F1 score of only 0.67, which is substantially lower than RAG's 0.85. These limitations motivate the investigation of knowledge-grounded approaches in which external security evidence is incorporated into the vulnerability-assessment process.

1.4.2 Effectiveness of RAG, Supervised Fine-Tuning, and Multi-Agents in Vulnerability Detection

An empirical evaluation by Saju et al. [18]. Provides a rigorous head-to-head comparison of three LLM-based vulnerability detection approaches against a baseline model. The study evaluated RAG, Supervised Fine-Tuning (SFT), and a Dual-Agent framework on a curated dataset focusing on five critical CWE categories (CWE-119, CWE-399, CWE-264, CWE-20, and CWE-200) compiled from Big-Vul and real-world GitHub repositories.

In the experimental setting reported by Saju et al. [19], RAG achieved the highest overall accuracy of 0.86 and an F1-score of 0.85 among the evaluated approaches, outperforming the reported SFT and Dual-Agent configurations.

The Dual-Agent system, while showing promise in improving “reasoning” transparency and error mitigation, did not match RAG's raw performance, These findings provide empirical motivation for retrieval-based vulnerability analysis, while their applicability to other datasets, models, and retrieval representations requires independent evaluation.

1.5 Knowledge Base Construction

1.5.1 Vulnerability Knowledge Structuring

The effectiveness of a RAG system is fundamentally tied to the quality and structure of its knowledge base.

Vul-RAG's first phase is dedicated to constructing this base by using an LLM to extract multi-dimensional knowledge from existing CVE instances. The key insight from Vul-RAG is that the knowledge must be high-level and generalizable, moving beyond simple code-near snippets to encompass functional semantics, root causes, and remediation strategies.

This approach ensures that the retrieved knowledge can be effectively applied to new, unseen code that shares semantic similarities with known vulnerabilities, rather than just lexical ones [5].

1.5.2 Vulnerability Representation

The representation of vulnerability knowledge directly impacts retrieval effectiveness. Vul-RAG's emphasis on high-level, generalizable knowledge is a direct response to the failure of lexical similarity to capture the subtle semantic differences between vulnerable and patched code. The knowledge extracted by Vul-RAG covering functional semantics, root causes, and fixing solutions—provides a more abstract and transferable representation than raw code alone. This allows the retriever to identify relevant knowledge based on the functional semantics of the code snippet under analysis, a more robust approach than relying on surface-level textual similarity [5].

Vulnerability knowledge can also be represented directly through source code rather than exclusively through abstract semantic descriptions. Code-oriented representations preserve implementation details that may distinguish vulnerable and secure variants, whereas higher-level descriptions can improve abstraction across syntactically different implementations. Consequently, the appropriate representation depends on the retrieval objective. The present work evaluates both perspectives by comparing functional-description retrieval with BGE-M3 and direct source-code retrieval with UniXcoder.

1.6 Tree-sitter-Based Structural Source-Code Extraction

Tree-sitter is a parser-generator tool and incremental parsing library designed to construct a concrete syntax tree from source code [8]. Unlike an unstructured sequence of characters or tokens, a concrete syntax tree represents the hierarchical organization of a file according to the grammar of its programming language. Tree-sitter generates a language-specific parser from a grammar and uses that parser to produce a tree representing the syntactic constructs present in the source file. It can also update an existing tree efficiently when the underlying source text changes, although the present project uses Tree-sitter primarily for structural parsing and extraction. Although Tree-sitter supports incremental reparsing, the present system uses it primarily for structural parsing and source-code unit extraction. [8].

The principal Tree-sitter abstractions are the language, parser, syntax tree and syntax node. A language object defines how a particular programming language is parsed, while a parser configured with that language produces a syntax tree for a supplied source file. The resulting tree consists of interconnected syntax nodes representing constructs such as declarations, identifiers, parameters, expressions, statements and blocks [8]. The node hierarchy therefore provides a structured representation of how individual source elements are nested within larger program constructs.

Tree-sitter produces a concrete syntax tree, rather than a conventional abstract syntax tree. A concrete syntax tree retains grammar-level details associated with the source representation, whereas an abstract syntax tree generally removes some concrete syntactic elements and expresses the program through a more abstract structural representation. Consequently, the terms *Tree-sitter parse tree*, *syntax tree* and *concrete syntax tree* are used in this report when referring specifically to Tree-sitter output.

1.6.1 Structural queries and source localization

Each Tree-sitter syntax node records its start and end positions in the original source code, together with its relationships to parent, child and sibling nodes [8]. These positions may be represented through byte offsets and row–column coordinates. The location information makes it possible to recover the exact source text covered by a matched node and to associate an extracted construct with its original line range.

Tree-sitter also provides a structural query language for identifying selected node configurations. A query is composed of one or more S-expression patterns that match node types and, where necessary, their children and named fields [8]. Captures may be assigned to matched nodes so that the application can retrieve the complete construct, its name or another relevant component. For example, the official documentation demonstrates a Python query that matches a function_definition node and captures its identifier as the declared function name. It also demonstrates that JavaScript function definitions may require different patterns, including patterns for functions assigned through assignment expressions [8].

The need for language-specific patterns is significant because equivalent programming concepts are not necessarily represented by identical node types in different grammars. A Python function may be represented by a function_definition node, whereas JavaScript and TypeScript grammars may distinguish function declarations, function expressions, arrow functions and method definitions. Tree-sitter queries therefore locate constructs through the grammar of the relevant language rather than through a universal function pattern [8]. The exact query definitions or traversal rules used by the present prototype are unspecified; accordingly, no specific implementation pattern beyond the documented extraction behavior is assumed here.

1.7 Retrieval Mechanisms and Vector Databases

The retrieval mechanism at the heart of a RAG system fundamentally determines whether relevant security knowledge reaches the LLM for analysis. Two distinct paradigms exist for this retrieval: lexical search and semantic search. Understanding their respective strengths and limitations is useful when selecting a retrieval strategy for source-code vulnerability analysis.

1.7.1 Lexical Search

Lexical retrieval represents queries and candidate records using discrete terms. BM25, one of the most established lexical ranking functions, estimates relevance from query-term occurrence, collection-level term frequency, and document-length-normalized term frequency. This makes lexical retrieval particularly suitable when exact identifiers or distinctive technical terms are important. Its effectiveness may nevertheless decrease when the query and a relevant record express the same concept using different vocabulary. Dense retrieval addresses this limitation by representing queries and records in a learned continuous vector space [14]. The relative effectiveness of these approaches is task-dependent, and heterogeneous evaluation has shown BM25 to remain a robust retrieval baseline. Despite its precision, lexical search suffers from a principal limitation when applied to code vulnerability detection which is vocabulary mismatch. Semantically related queries and documents may receive weak scores when they express the same concept using different terminology.

1.7.2 Semantic Search

Semantic search addresses the limitations of lexical matching by encoding both queries and documents into high-dimensional vector representations (embeddings) stored in a vector database. Semantic search represents queries and candidate records as dense vectors and ranks candidates according to a vector-space similarity or distance measure.

Dense retrieval can reduce vocabulary mismatch by ranking records through learned representational proximity rather than exact term overlap [14]. Nevertheless, dense retrieval is not universally superior to lexical retrieval. Results from the BEIR benchmark show that retrieval effectiveness varies across domains and that BM25 remains a robust zero-shot baseline [19]. Dense representations must therefore be evaluated on the intended retrieval task, and similarity values should be interpreted as ranking quantities rather than direct evidence of factual relevance.

1.7.3 Vector Database

A vector store maintains embedding vectors together with their associated records and metadata and exposes similarity- or distance-based nearest-neighbour queries. Its role is to operationalize retrieval over representations produced by the embedding model.

1.7.4 ChromaDB

Chroma is a vector-storage system that supports persistent collections of embeddings, associated documents and metadata, and nearest-neighbour querying over dense vector representations [15]. In single-node deployments, Chroma uses an HNSW index for approximate nearest-neighbour search and supports configurable vector-space functions such as cosine, inner-product and squared Euclidean distance [15]. Chroma reports distance-oriented retrieval scores, under which smaller values represent closer matches for the configured distance function. These scores quantify vector-space proximity and should not be interpreted as probabilities of relevance or downstream classification confidence.

Core features of ChromaDB:

- Lightweight and Embedded Architecture: ChromaDB's core is a Python library that can be directly integrated into application code without requiring a standalone server deployment (though client-server mode is also supported). It installs with a single pip install chromadb command and can run embedded within the same Python process. This design enables developers to build and iterate rapidly, embedding ChromaDB into local development environments, evaluation pipelines, or applications without external infrastructure or cloud dependency.
- HNSW Indexing: ChromaDB uses the Hierarchical Navigable Small World (HNSW) algorithm for efficient approximate nearest neighbor (ANN) similarity search. HNSW constructs a multi-layered graph structure where each layer provides increasingly coarse navigation, enabling logarithmic-time search complexity for high-dimensional vectors.
- Metadata Filtering: ChromaDB supports structured metadata alongside stored vectors and permits retrieval to be restricted using metadata conditions. Although this capability can support language- or category-specific retrieval, the evaluated configurations in this project rank candidates primarily according to their vector representations.
- Persistent Storage: ChromaDB provides persistent storage through configurable backends, ensuring that the knowledge base survives process restarts and can be incrementally updated without full re-indexing.
- Simple Python API: ChromaDB offers a deliberately minimal API surface. Installing it as a Python package, embedding documents, and querying requires only a few lines of code, making it the fastest path from zero to a working prototype. For MVPs and evaluation systems, nothing is faster to set up.

1.8 Evaluation Methodologies & Benchmarks

1.8.1 Evaluation Metrics for Vulnerability Detection

Evaluating vulnerability detection systems requires a nuanced approach that goes beyond simple accuracy. The use of Precision, Recall, and F1-Score is standard, but the choice of metric weighting can reflect different stakeholder priorities.

For instance, RealVuln, a benchmark for real-world code, uses the F3 score (beta=3, which weights recall nine times more than precision) to prioritize finding as many vulnerabilities as possible, a common concern for security teams [20].

The SecLens-R framework further refines this by introducing a multi-stakeholder evaluation with 35 shared dimensions and five role-specific weighting profiles (CISO, Chief AI Officer, Security Researcher, Head of Engineering, and AI-as-Actor), revealing that the same model's "Decision Score" can vary by as much as 31 points depending on the stakeholder perspective [21].

1.8.2 Dedicated Vulnerability Benchmarks

Dedicated vulnerability benchmarks provide standardized datasets and ground-truth labels for assessing the effectiveness of automated security-analysis systems. Their role is particularly important in vulnerability detection because reported performance can vary substantially depending on the realism of the test cases, the granularity of the analysis unit, the balance between vulnerable and non-vulnerable samples, and the way findings are matched to expected weaknesses.

The OWASP Benchmark project provides purpose-built security test cases with documented expected results for evaluating vulnerability-detection tools. Its Python variant, OWASP BenchmarkPython, contains both vulnerable and non-vulnerable test cases associated with expected weakness categories and CWE identifiers [22]. This structure supports reproducible evaluation of binary vulnerability detection as well as CWE classification and makes it suitable for controlled comparison among different analysis approaches under a common ground truth.

More recent benchmarks attempt to address limitations of isolated or synthetic function-level evaluation by introducing repository context, real-world vulnerability histories, or more complex analysis settings. RealVuln evaluates multiple scanners on intentionally vulnerable Python repositories using hand-labeled findings, with the aim of assessing how rule-based, general-purpose LLM, and security-specialized tools behave on realistic repository-level code [20]. JITVUL focuses on just-in-time vulnerability detection by linking vulnerable functions to vulnerability-introducing and vulnerability-fixing commits across a large collection of CVEs, thereby supporting evaluation under software-evolution scenarios [23]. SecLLMHolmes provides a broader automated framework for evaluating LLM-based vulnerability-detection systems across multiple datasets and reasoning settings [24]. ZeroDayBench extends evaluation toward agent-based systems by examining whether LLM agents can identify and remediate previously unseen vulnerabilities in open-source projects [25].

These benchmarks differ in scope and should therefore not be treated as directly interchangeable. Repository-level benchmarks provide greater contextual realism, but they also introduce more complex matching rules, interprocedural dependencies, and annotation requirements. Function-level benchmarks offer a more controlled setting for measuring detection and classification performance but may not fully reflect the complexity of real-world software.

For the present work, OWASP BenchmarkPython v0.1 was selected as the primary evaluation benchmark because its structure aligns closely with the implemented analysis unit and research objective. The proposed auditor performs function-level reasoning, and OWASP BenchmarkPython provides 1,230 labelled Python test cases, including both vulnerable and non-vulnerable examples, together with expected CWE information. This allows the two RAG retrieval configurations—BGE-M3 functional-semantic retrieval and UniXcoder code-oriented retrieval—to be evaluated under the same ground truth and compared directly with Bandit using precision, recall, F1-score, accuracy, specificity, false-positive rate, MCC, and CWE classification accuracy. The benchmark is therefore appropriate for examining not only vulnerability coverage, but also the project's central concern of false-positive control under different retrieval representations.

1.8.3 Challenges in Real-World Evaluation

Evaluating vulnerability-detection systems on realistic software remains challenging because benchmark design must balance reproducibility, language coverage, annotation quality, vulnerability diversity, and similarity to real development environments. Many existing datasets evaluate isolated functions or relatively simple test cases containing a single dominant weakness, whereas production software may contain several independent vulnerabilities within the same file [26].

Research on multi-vulnerability detection illustrates this issue. As shown in Figure 3, model performance decreases as vulnerability density increases, while recall also varies across programming languages [26]. These findings indicate that evaluation results may depend not only on the detection model itself, but also on the number of vulnerabilities present in the analyzed code and on the characteristics of the programming language used in the benchmark.

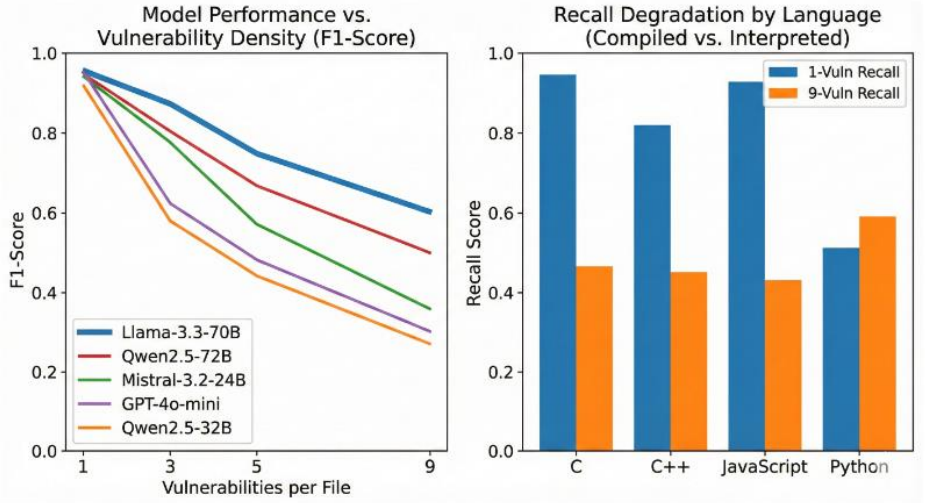

Figure 3 - Evaluation of LLMs on Code Vulnerability Detection Across Varying Density and Language Types *[26]*.

Benchmark construction introduces an additional limitation. Some datasets rely on synthetically introduced vulnerabilities rather than exclusively on naturally occurring security defects. Synthetic generation is useful because it enables controlled experiments and systematic vulnerability coverage, but it may also introduce patterns or coding artefacts that differ from naturally occurring vulnerabilities [26]. Therefore, results obtained on synthetic or semi-synthetic benchmarks should be interpreted within the scope of the corresponding evaluation setting.

Another practical challenge concerns the availability of suitable benchmarks across programming languages. The implemented auditing framework supports Python, JavaScript, TypeScript, and TSX; however, a quantitative comparison of the retrieval configurations requires a benchmark with reliable vulnerable and non-vulnerable labels, sufficiently clear CWE ground truth, adequate sample size, and a structure suitable for consistent automated scoring. During the benchmark-selection process, no JavaScript or TypeScript benchmark was identified that satisfied these requirements to the same extent as the selected Python benchmark.

Using different benchmarks for different programming languages would also introduce additional variation in dataset construction, vulnerability distribution, and scoring criteria, making direct comparison of the retrieval approaches less controlled. For this reason, the quantitative evaluation was conducted using OWASP BenchmarkPython v0.1, which provides a consistent labelled test set and expected CWE information for all evaluated approaches. This choice allows BGE-M3 retrieval, UniXcoder retrieval, and Bandit to be assessed under identical ground-truth conditions.

Accordingly, the experimental results reported in this work are limited to the Python benchmark setting. The broader implementation nevertheless retains support for JavaScript, TypeScript, and TSX at the preprocessing and auditing levels. Extending the quantitative evaluation to these

languages requires suitably labelled and comparable vulnerability benchmarks and is therefore left for future evaluation work.

1.8.1 Evaluation Metrics for Vulnerability Detection

The evaluation of vulnerability-detection systems should consider both their ability to identify genuine vulnerabilities and their tendency to incorrectly flag non-vulnerable code. This distinction is particularly important in security auditing because a detector that reports most vulnerable cases but also generates a large number of false alarms may impose substantial manual review effort. The OWASP Benchmark evaluation methodology therefore distinguishes four fundamental outcomes: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), and uses them to characterize both vulnerability coverage and false-positive behavior [22].

Precision measures the proportion of reported vulnerabilities that are actually vulnerable, whereas recall, also referred to as sensitivity or true-positive rate, measures the proportion of ground-truth vulnerabilities that are successfully detected. These metrics represent complementary properties of a vulnerability detector: precision reflects the reliability of positive findings, while recall reflects vulnerability coverage. The F1-score combines precision and recall using their harmonic mean and is therefore commonly used when a balanced summary of both measures is required.

However, recall-oriented measures alone do not characterize how well a system handles non-vulnerable code. Specificity, defined as , measures the proportion of non-vulnerable cases that are correctly rejected, whereas the False Positive Rate (FPR), defined as , measures the proportion of non-vulnerable cases that are incorrectly reported as vulnerable. OWASP Benchmark explicitly incorporates false-positive behavior into its scoring philosophy because a detector that flags nearly every case may achieve high recall while providing little practical value [22]. Accuracy measures the proportion of all correctly classified cases, but it should be interpreted together with class-sensitive metrics rather than in isolation.

For the present work, this multi-metric evaluation is particularly important because the objective is not only to detect vulnerabilities but also to examine how retrieval representation influences false-positive control. The two RAG configurations are therefore evaluated using precision, recall, F1-score, accuracy, specificity, false-positive rate. CWE accuracy is additionally calculated to determine whether correctly detected vulnerabilities are assigned an appropriate weakness category. Together, these measures allow the UniXcoder and BGE-M3 retrieval configurations to be compared in terms of vulnerability coverage, positive-prediction reliability, rejection of non-vulnerable cases, and weakness classification.

1.9 Challenges in Vulnerability Dataset Quality

1.9.1 Label Inaccuracy and Data Duplication

A primary obstacle in the development of automated security auditors is the low quality of existing training data. Empirical analysis of seven major vulnerability datasets reveals that widely used

resources suffer from label inaccuracy rates ranging from 20% to 71%. Furthermore, data duplication is a pervasive issue; rigorous deduplication processes have shown that up to 66.94% of vulnerable-fixed pairs in common datasets are redundant or "self-identical," meaning the vulnerable code is identical to its fixed version. This lack of a clear "learning signal" in self-identical pairs prevents models from understanding the precise semantic changes required to mitigate a flaw. For example, the BigVul dataset, frequently used in research, was found to have a validity rate of only 25%, with 94.4% of its pairs providing no meaningful distinction between vulnerable and secure states [27].

1.9.2 Generalization Gap

In the context of LLM-based vulnerability detection, a significant "generalization gap" exists, defined as the performance difference between In-Distribution (ID) testing (using the same dataset as training) and Out-of-Distribution (OOD) testing (using independent, unseen real-world data). Research indicates that ID performance is a poor predictor of real-world effectiveness. Models often achieve misleadingly high ID scores by memorizing dataset-specific artifacts and spurious correlations rather than learning genuine vulnerability patterns. A stark example of this is the Qwen2.5-Coder model, which achieved a high 0.703 ID accuracy on BigVul but collapsed to a 0.493 OOD accuracy on real-world samples—performance essentially no better than random guessing. These findings motivate approaches that supplement parametric model knowledge with explicit external evidence; however, retrieval augmentation must itself be evaluated because its effectiveness depends on knowledge quality and retrieval relevance [27].

1.9.3 The Pitfall of “Vulnerability-Like” Code Recognition

A critical theoretical failure mode in many detection models is the "vulnerability-like" code detection pitfall. This occurs when models are trained on datasets that pair vulnerable code with general benign code from the same repository rather than their corresponding fixed versions. While these models achieve high accuracy (often >0.96) at identifying code that "looks" like a vulnerability, they fail to distinguish between a flaw and its specific fix [27]. This suggests the model has not learned the precise, semantic nature of the vulnerability itself. To be effective, an auditor must be able to recognize the subtle textual differences—such as relocated method invocations or modified conditional checks—that separate a risk from a remediation [5].

1.9.4 Data Sparsity and Class Imbalance

The vulnerability landscape is characterized by severe class imbalance, which creates a theoretical bottleneck for detecting rare but dangerous weaknesses. In consolidated vulnerability databases, the top 5–10 CWEs typically comprise 55% to 80% of all available samples. The frequency ratio between the most common type (e.g., CWE-20: Improper Input Validation) and the least common (e.g., CWE-798: Use of Hard-coded Credentials) can be as high as 130:1. This data sparsity directly impacts model performance; models trained on imbalanced data perform poorly on "long-tail" CWE categories, regardless of their architectural sophistication. This justifies the use of

synthetic data generation (RVG) to augment rare categories, which has been shown to improve detection accuracy on real-world data by 5.8% to 13.1% [27].

1.10 Practical Implementation Considerations

1.10.1 Lightweight vs. Heavyweight Architectures

The comparison by Saju et al. [18]. Provides motivation for the lightweight retrieval-oriented design adopted in the present work which emphasizes a lightweight and practical design, as the RAG approach achieved the highest performance without the orchestration complexity of a multi-agent system or the retraining cost of fine-tuning.

The token efficiency of RAG, as demonstrated by the comparison with RCI, further underscores its suitability for a practical, resource-conscious auditor [28].

1.10.2 Explainability and Traceability

RAG-based systems can improve traceability because the external records retrieved for a particular input can be retained and inspected as supporting evidence. Vul-RAG's generated knowledge served as high-quality explanations that improved manual detection accuracy from 60% to 77%, demonstrating the practical value of this traceability [5].

1.10.3 Continuous Knowledge Updates

One of the most significant practical benefits of RAG is that it enables continuous updates to the system's knowledge and delivers precise, current responses by extracting relevant items from a vector database without requiring expensive and time-consuming model retraining.

This is critical for maintaining relevance against the rapidly evolving landscape of newly disclosed CVEs, a challenge that frameworks like AutoPatch address by using RAG with a structured database of recently disclosed vulnerabilities [29].

1.11 Conclusion

This chapter established the theoretical foundation of the proposed RAG-based source-code security auditing system. It examined the role of Large Language Models in code understanding and vulnerability analysis, while highlighting the limitations of relying exclusively on knowledge encoded within model parameters. It also introduced Retrieval-Augmented Generation as a mechanism for incorporating external security knowledge at inference time, thereby supporting more informed and contextually grounded model reasoning.

The chapter further discussed the principal components required to construct such a system, including structural source-code extraction, dense vector representations, external knowledge storage, similarity-based retrieval, and structured LLM-based vulnerability assessment. Particular attention was given to the distinction between retrieval relevance and vulnerability judgement:

retrieved records provide contextual evidence, whereas the final security decision requires examination of the analysed source code itself.

These concepts establish the theoretical basis for the two retrieval configurations evaluated in this work. Functional-semantic retrieval with BGE-M3 abstracts source code through natural-language descriptions, whereas code-oriented retrieval with UniXcoder preserves implementation-level representations. Their comparison provides a basis for examining how retrieval representation affects vulnerability detection, CWE classification, and particularly false-positive control within a common RAG-based auditing architecture.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews prior research on automated software vulnerability detection and AI-assisted security auditing, with particular focus on recent Large Language Model (LLM)–based approaches. Across the surveyed works, a consistent motivation emerges: security flaws can be subtle and context-dependent, while the scale and rapid evolution of modern software make exhaustive manual auditing difficult. As a result, many studies explore how LLMs can support vulnerability identification, explanation generation, and even remediation. However, empirical evidence indicates that LLM-based security judgments can be sensitive to prompt design, training data scope, and the availability of external evidence, which is especially problematic in security-critical settings where incorrect conclusions can create false assurance or unnecessary triage effort [1]. The following review is organized thematically to compare how prior work operationalizes LLMs for security: (i) prompt-only inspection, (ii) fine-tuning and supervised adaptation, (iii) agent-based and multi-agent workflows, (iv) retrieval-augmented and knowledge-grounded systems, (v) knowledge-base construction and vulnerability representation, and (vi) evaluation practices and benchmarks. Collectively, these themes motivate the design of the proposed RAG-based Security Auditor, with particular emphasis on evidence-grounded reasoning, retrieval representation, structured vulnerability assessment, and the practical control of false-positive findings.

2. LLM-only prompting for security code inspection

A foundational baseline in the contemporary literature is prompt-based security inspection, where a general-purpose GPT-style model is given source code and asked to identify vulnerabilities and propose fixes. “A New Approach to Web Application Security: Utilizing GPT Language Models for Source Code Inspection” exemplifies this direction by framing the LLM as an end-to-end analyzer driven primarily by natural-language instructions [3]. The key advantage of prompt-only inspection is low engineering overhead: it requires no dedicated training pipeline and can be applied quickly across different code artifacts. Yet the same simplicity exposes important limitations. Because the model’s decision is primarily conditioned on the prompt and its internal knowledge, reliability can vary substantially across prompt formulations and across vulnerability types, and the output may lack stable provenance when the model does not explicitly ground claims in external vulnerability evidence [18], [3]. This baseline therefore motivates subsequent work that strengthens LLM-based auditing through either task-specific adaptation (fine-tuning), process modeling (agents), or external knowledge grounding (retrieval).

2.3 Fine-tuning and supervised adaptation for secure code review

A second line of research attempts to improve LLM performance by adapting models to security tasks through supervised fine-tuning (SFT) or security-aware training. SecureReviewer explicitly proposes “secure-aware fine-tuning” to enhance LLM behavior in secure code review scenarios, reflecting the view that generic pretraining alone does not consistently yield security-correct

judgments across diverse code contexts [30]. RealVul similarly investigates whether LLMs can detect vulnerabilities in web applications, representing a task- and dataset-driven approach to improving detection through targeted training and evaluation [31]. Compared with prompt-only inspection, fine-tuning can improve consistency because a model is optimized for the target task; however, the literature also highlights that fine-tuning introduces stronger dependence on dataset coverage and labeling quality, and it can be costly to update as vulnerability patterns and best practices evolve [18], [30]. This trade-off is important for the project because a practical auditor must remain maintainable over time, motivating solutions that reduce the need for frequent retraining while still improving reliability.

2.4 Agent-based and multi-agent auditing workflows

Because real security auditing is iterative—often requiring context gathering, hypothesis testing, and validation—several works model vulnerability analysis as an explicit workflow rather than a single prediction. RepoAudit exemplifies this process-oriented view by proposing an autonomous LLM agent for repository-level code auditing, shifting attention from isolated functions toward codebase exploration and multi-step reasoning across files and components [32]. This approach is motivated by the observation that vulnerabilities frequently depend on broader program context (e.g., how inputs propagate or how components interact), which single-shot prompting may not capture reliably. Multi-agent research extends this workflow perspective by decomposing the auditing task into specialized roles or cooperative behaviors. MulVul proposes a retrieval-augmented multi-agent system, indicating that distributing responsibilities across agents can help address heterogeneity in vulnerability patterns and can enable more structured exploration and refinement during detection [33]. AutoPatch further demonstrates a multi-agent architecture in a closely related setting—patching real-world CVEs generated by outdated LLMs—suggesting that agent coordination can support complex, multi-step security workflows beyond detection alone [29]. At the same time, agent-based systems introduce practical concerns: orchestration complexity, higher computational overhead, and difficulty scaling workflows to large repositories without careful control of context and verification steps [18], [33], [32]. These limitations motivate the project to seek a lighter-weight auditing pipeline that still supports strong evidence grounding and structured reporting.

2.5 Retrieval-augmented and knowledge-grounded vulnerability analysis

A major contemporary trend is to ground LLM reasoning in external security knowledge retrieved at inference time. The empirical evaluation in [18] frames Retrieval-Augmented Generation (RAG) as a primary family of approaches alongside SFT and dual-agent systems, reflecting the growing consensus that retrieval can mitigate limitations associated with static parametric knowledge and prompt sensitivity [18]. Vul-RAG is particularly relevant to this project because it proposes a knowledge-level RAG framework for vulnerability detection, emphasizing that retrieving distilled vulnerability knowledge (rather than only code-near snippets) can guide an LLM to reason about vulnerability behavior, causes, and fixes more effectively [5]. In parallel,

ReVul-CoT proposes combining retrieval augmentation with structured reasoning prompts (Chain-of-Thought), indicating that retrieval alone may be insufficient unless coupled with a disciplined reasoning process that uses retrieved evidence to justify conclusions [17]. RAG also appears in multi-agent settings: MulVul integrates retrieval into a multi-agent detection pipeline, and AutoPatch uses a multi-agent framework to patch vulnerabilities that arise due to outdated LLM knowledge, both reinforcing the view that retrieval is increasingly treated as a practical mechanism for keeping security reasoning aligned with evolving vulnerability information [33], [29].

Although prior RAG-based vulnerability-detection studies demonstrate the value of external security knowledge, they also show that retrieval effectiveness depends on how the query and knowledge records are represented. Existing approaches commonly emphasize functional semantics, vulnerability causes, fixing strategies, or task-specific knowledge structures [5], [17]. The present work extends this line of investigation by comparing two alternative dense retrieval representations under the same downstream reasoning process: functional-semantic retrieval using BGE-M3 and direct source-code retrieval using UniXcoder. This comparison isolates retrieval representation as an important design factor in determining which security evidence is supplied to the reasoning model.

2.6 Knowledge-Base construction and vulnerability representation

Across retrieval-centered systems, multiple studies emphasize that “what” is retrieved matters as much as “how” it is retrieved. Vul-RAG motivates multi-dimensional vulnerability knowledge—including functional semantics, root causes, and fixing solutions—suggesting that abstract, generalizable representations can guide auditing more effectively than narrow lexical similarity [5]. ReVul-CoT similarly foregrounds building a local knowledge base that fuses multiple vulnerability information sources, aligning with the argument that richer, structured knowledge can improve both decision quality and interpretability when used as retrieved context during assessment [17]. AutoPatch reinforces the operational importance of maintainable vulnerability knowledge by explicitly targeting real-world CVEs and highlighting the practical risk of outdated model knowledge, which retrieval-based designs attempt to address without continuous retraining [29].

The literature therefore suggests that vulnerability knowledge can be represented at different levels of abstraction. High-level semantic descriptions may improve transfer across syntactically different implementations, whereas direct source-code representations preserve implementation details that may be security-critical. These two perspectives are not necessarily interchangeable. The present system therefore evaluates both by using BGE-M3 over generated functional descriptions and UniXcoder over raw source code, allowing their effect on downstream vulnerability detection to be compared within a common knowledge-base and reasoning architecture.

2.7 Evaluation and benchmarking practices

Credible security tooling depends on evaluation protocols that reflect real-world difficulty and operational constraints. The empirical comparison in [18] highlights the need to evaluate multiple methodological families (RAG, SFT, and dual-agent or agent-assisted settings) under consistent criteria, making explicit that reported performance and practical reliability can vary substantially across approach classes [18]. RealVul and SecureReviewer represent task-focused studies that evaluate LLM performance under curated settings for vulnerability detection and secure review, providing evidence that targeted adaptation can improve outcomes but may also bind performance to dataset scope and update frequency [31], [30]. Vul-RAG and ReVul-CoT highlight evaluation settings where retrieval and reasoning strategies are tested as part of the overall detection/assessment pipeline, reinforcing that the "system" (retrieval + reasoning + reporting) is often the unit of evaluation rather than the LLM alone [5], [17]. For the project, these evaluation practices motivate designing an auditor that can be assessed not only on detection correctness but also on explanation quality, mapping accuracy (e.g., to weakness/vulnerability references), and the practical usefulness of remediation recommendations.

For the present project, these evaluation practices motivate a controlled comparison in which the two RAG retrieval configurations are evaluated on the same labelled benchmark and under the same downstream reasoning stage. Detection performance is measured using confusion-matrix-based metrics, including precision, recall, F1-score, accuracy, specificity, false-positive rate, and MCC, while CWE accuracy is evaluated separately for correctly detected vulnerable cases. Bandit is included as a conventional static-analysis baseline to provide a reproducible comparison with a non-LLM security tool.

Table 1- Previous works comparison

Ref.	Objective and analysis unit	Core method	Knowledge and context handling	Output	Evaluation setting	Main reported results
[5]	Function-level vulnerability detection, with emphasis on distinguishing vulnerable functions from their patched counterparts.	Knowledge-level RAG. An LLM extracts functional semantics, vulnerability causes, and fixing solutions; BM25 retrieval and reciprocal-rank fusion identify related knowledge for LLM reasoning.	Uses historical vulnerable–patched CVE pairs. Analysis remains function-level; it does not construct repository-wide control- or data-flow relations.	Vulnerable/benign decision with vulnerability-oriented explanatory knowledge.	PairVul: 586 vulnerable–patched function pairs from 420 Linux-kernel CVEs; user study; case study on Linux kernel v6.9.6.	Improved pairwise accuracy by 16–24 percentage points over the evaluated base LLMs; improved balanced precision by 9–14 points and balanced recall by 7–11 points. Explanatory knowledge increased manual confirmation accuracy from 60% to 77%. The case study found 10 previously unknown bugs, six of which received CVE identifiers.

Ref.	Objective and analysis unit	Core method	Knowledge and context handling	Output	Evaluation setting	Main reported results
[30]	Security-focused code-review comment generation for pull-request code changes.	Secure-aware LoRA fine-tuning combined with retrieval-augmented review generation. BM25 retrieves a security-review template after the issue type is predicted.	Uses code diffs and a datastore of 261 review templates derived from the training set. The method does not perform repository-level program analysis.	Security type, issue description, impact, and actionable remediation advice; includes a Non-Issue class.	A curated dataset of 4,674 examples covering seven security categories plus Non-Issue; 300-example test set and human evaluation.	The best configuration achieved a 71.98% F1 score for issue detection and a SecureBLEU score of 29.31. Relative to the strongest baseline, the paper reports approximately 17% higher F1, 18% higher accuracy, and 19% higher SecureBLEU. SecureBLEU correlated strongly with human judgement (Pearson r = 0.7533).
[32]	Autonomous repository-level detection of path-sensitive data-flow bugs.	LLM agent with initiator, explorer, memory, and validator modules; demand-driven traversal follows relevant callers, callees, data-flow facts, and feasible paths.	Uses repository source code rather than an external vulnerability knowledge base. It explicitly explores intraprocedural context and validates path conditions.	Bug reports containing bug type, source locations, data-flow facts, and feasible-path evidence.	Fifteen real-world benchmark projects for null-pointer dereference, memory leak, and use-after-free; additional scans of actively maintained repositories.	Detected 40 true bugs with 11 false positives, yielding 78.43% precision; 21 findings were intraprocedural. Average analysis cost was 0.44 hours and US$2.54 per project. In the broader deployment study, 185 new bugs reported, of which 174 were confirmed or fixed.
[31]	Snippet-level detection of PHP web vulnerabilities: CWE-79 (XSS) and CWE-89 (SQL injection).	Vulnerability-trigger localization, AST-based control/data-flow slicing, normalization, synthetic-data generation, and separate fine-tuned code models for each vulnerability type.	No retrieval component. Context is represented through vulnerability-oriented slices rather than complete functions or repository-wide reasoning.	Binary vulnerable/benign classification for a specific CWE.	Vulnerability data from 180 PHP projects; random-split and unseen-project experiments; comparison with vulnerability-repair sampling and PHP SAST tools.	On random samples, the best F1 scores were 83.68% for CWE-79 (CodeLlama-7B) and 79.37% for CWE-89 (StarCoder2-3B), whereas the repair-based baseline remained at or below 50% F1. On unseen projects, the best F1 scores were 69.86% for CWE-79 and 75.00% for CWE-89.
[3]	GPT-assisted static inspection of inadequate isolation of sensitive data (CWE-653) in Angular applications.	A multi-stage prompt pipeline using GPT-3.5 and GPT-4, few-shot examples, and structured JSON intermediates; source files are processed in prompt-sized fragments.	No external retrieval or fine-tuning. Project information is accumulated across staged prompts, but the method is specialized to Angular and CWE-653.	Sensitive-data identification, sensitivity and protection classifications, and final CWE-653 findings.	Manual comparison of 12 open-source Angular projects.	The paper reports an 88.76% vulnerability-detection rate for GPT-4 in its CWE-653 evaluation. Results are specific to the authors' sensitivity/protection taxonomy and the selected Angular projects.

The comparison in Table 1 illustrates several distinct directions in LLM-assisted software vulnerability analysis. Taken together, the reviewed studies show that LLM-assisted security analysis can be improved through several complementary strategies: domain-specific fine-tuning, vulnerability-oriented preprocessing, repository exploration, structured validation, and retrieval-augmented reasoning. Nevertheless, the systems differ substantially in their objectives and evaluation settings. SecureReviewer analyses code changes and generates review comments;

RepoAudit performs path-sensitive repository exploration for selected data-flow bugs; RealVul trains CWE-specific classifiers over sliced PHP snippets; Szabó and Bilicki examine a single weakness in Angular projects; and Vul-RAG evaluates function-level vulnerable–patched discrimination using multidimensional CVE knowledge. Their numerical results should therefore be interpreted within their respective tasks rather than compared as direct performance rankings.

The proposed system adopts a distinct combination of design choices. It accepts a user-supplied source-code directory and uses Tree-sitter to extract complete functions and methods from Python, JavaScript, TypeScript, and TSX files while preserving file paths and line ranges. Rather than relying on a single retrieval representation, the system implements two RAG configurations over the same prepared knowledge base. The first uses BGE-M3 to retrieve records through generated functional descriptions, while the second uses UniXcoder to retrieve records through direct source-code embeddings. In both configurations, the retrieved records are supplied with the original target function to GPT-5.1, which produces a structured verdict of vulnerable, not vulnerable, or uncertain together with CWE classification, affected source lines, technical explanation, and remediation guidance. The resulting findings are aggregated into an HTML security audit report.

Accordingly, the principal contribution of the proposed system lies not in repository-level program analysis or task-specific fine-tuning, but in integrating automated directory preprocessing, function-level structural extraction, external vulnerability knowledge, alternative retrieval representations, structured LLM-based reasoning, source-location preservation, and developer-oriented report generation within a unified auditing workflow. In contrast with prior work that generally evaluates a single retrieval formulation, the present study explicitly compares functional-semantic and code-oriented retrieval to examine how representation choice affects vulnerability detection and false-positive control. This positioning defines the system as a practical function-level auditing framework in which retrieval serves as an evidence-grounding mechanism rather than as a direct vulnerability classifier.

Chapter 3: Environment Setup

3.1 Introduction

This chapter presents the development environment and technical resources used to implement the proposed RAG-based source-code security auditor. Its purpose is to document the software stack, model services, parsing tools, persistent storage components, and configuration choices that support the implemented prototype. While the preceding chapters established the theoretical basis of the system, this chapter focuses on the technologies required to realize that design in a reproducible development environment. The detailed interaction among these components is presented later in the practical implementation chapter.

The environment supports two alternative retrieval configurations: functional-semantic retrieval using BGE-M3 and direct source-code retrieval using UniXcoder, while sharing the same structural preprocessing, vulnerability knowledge base, reasoning model, and reporting components.

3.2 Development Environment

The prototype was implemented in a Python-based development environment supporting an offline knowledge-base preparation phase and an online source-code auditing phase. Two retrieval configurations were prepared. In the BGE-M3 configuration, functional descriptions associated with the knowledge-base records are embedded using BAAI/bge-m3. In the UniXcoder configuration, the vulnerable source-code examples are embedded directly using microsoft/unixcoder-base. Both configurations use persistent ChromaDB collections so that the indexed knowledge base can be reused across multiple auditing runs.

3.2.1 Hardware Environment

The prototype was developed and tested on a local ASUS TUF Gaming F15 FX507ZV4/FX507ZV workstation. The system was equipped with a 12th-generation Intel Core i7-12700H processor operating at a base frequency of 2.30 GHz and 16 GB of installed RAM, of which approximately 15.6 GB was available to the operating system. The machine used a 64-bit x86-64 architecture.

The local workstation executed the principal non-generative stages of the system, including recursive directory traversal, Tree-sitter parsing, function and method extraction, BGE-M3 or UniXcoder embedding generation depending on the selected retrieval configuration, ChromaDB knowledge-base construction, nearest-neighbor retrieval, structured result aggregation, and HTML report generation.

3.2.2 Software Environment

The software environment was implemented primarily in Python, which serves as the central programming language for coordinating the system's preprocessing, retrieval, reasoning, and reporting stages. Python was selected because it provides mature libraries for source-code parsing, machine-learning inference, vector storage, API communication, structured data processing, and report generation. Tree-sitter is used for language-aware structural parsing of Python, JavaScript,

TypeScript, and TSX source files. The corresponding language grammars enable the system to identify classes, functions, and methods while preserving their file paths and source-line ranges. This parsing stage provides the function-level analysis units used by the later pipeline; it does not construct control-flow graphs, data-flow graphs, call graphs, or program slices.

The system implements two alternative embedding and retrieval configurations. In the first configuration, GPT-5-mini generates a concise natural-language functional description for each extracted function or method. These descriptions are encoded using BAAI/bge-m3 into dense vector representations, enabling functional-semantic retrieval. In the second configuration, the original source code is embedded directly using microsoft/unixcoder-base, providing a code-oriented representation without requiring an intermediate functional-description generation step. PyTorch provides the underlying local inference support for the embedding models.

For both configurations, the resulting knowledge-base embeddings are stored in persistent ChromaDB collections. During auditing, the analyzed function is represented using the same embedding strategy as the corresponding knowledge-base collection. The BGE-M3 configuration compares the generated functional description of the target function with stored functional-description embeddings, whereas the UniXcoder configuration compares the target source code directly with stored vulnerable-code embeddings. In each case, ChromaDB returns the three highest-ranked records, which are subsequently restored from the complete knowledge base and supplied as contextual evidence to the vulnerability-reasoning stage. ChromaDB therefore functions as the vector-storage and nearest-neighbour retrieval layer and does not itself perform vulnerability classification or CWE prediction.

The OpenAI Python SDK is used to access the language-model components of the system. GPT-5-mini is required only by the BGE-M3 configuration for functional-description generation, while GPT-5.1 is used by both retrieval configurations as the common downstream vulnerability-reasoning model. GPT-5.1 receives the original target function together with the retrieved knowledge records and produces a structured security assessment. Additional Python utilities are used for configuration management, environment-variable loading, intermediate JSON and JSONL processing, error handling, and report generation.

Together, these components form the technical environment of the prototype. Tree-sitter provides structural source-code extraction; BGE-M3 and UniXcoder provide alternative retrieval representations; ChromaDB provides persistent vector storage and nearest-neighbour retrieval; GPT-5-mini supports functional-description generation in the BGE-M3 configuration; GPT-5.1 performs the final vulnerability assessment; and the local Python reporting component transforms the structured findings into user-facing HTML and JSON security audit reports.

Table 2 - Software Components Used in the Development Environment

Software component	Role in the system

Python	Core implementation language and coordination of preprocessing, retrieval, reasoning, and reporting stages
Tree-sitter	Language-aware structural parsing of supported source files
Python Tree-sitter grammar	Parsing Python classes, functions, and methods
JavaScript Tree-sitter grammar	Parsing JavaScript functions, methods, and classes
TypeScript/TSX Tree-sitter grammar	Parsing TypeScript and TSX source constructs
microsoft/unixcoder-base	Direct source-code embedding and code-oriented retrieval in the UniXcoder RAG configuration
BAAI/bge-m3	Dense semantic representation of functional descriptions in the BGE-M3 retrieval configuration.
PyTorch	Local inference for embedding generation
ChromaDB	Persistent storage of embeddings, documents, and metadata; nearest-neighbour retrieval
OpenAI Python SDK	Communication with GPT-5-mini and GPT-5.1 through the OpenAI API
GPT-5-mini	Generation of functional descriptions for the BGE-M3 retrieval configuration.
GPT-5.1	Structured vulnerability assessment using source code, descriptions, and retrieved records

3.3 Tools and Libraries

This section presents the principal tools and libraries used to implement the proposed RAG-based source-code security auditor. The selected components support the main stages of the system, including source-directory traversal, language-aware parsing, knowledge-base processing, functional-description generation, dense embedding generation, vector storage and retrieval, structured vulnerability assessment, and HTML report production. Each component is assigned a specific role within the pipeline, allowing the prototype to remain modular without requiring local training or fine-tuning of a Large Language Model.

3.3.1 Python and Project Utilities

Python was selected as the primary implementation language for the prototype because it provides well-established libraries for source-code processing, machine-learning inference, vector retrieval, API communication, structured data handling, and report generation. In the implemented system, Python directly coordinates the individual processing stages through custom modules rather than through an external orchestration framework such as LangChain.

Standard Python utilities are used to support project-level operations. Directory traversal and file handling are performed through modules such as pathlib and os, enabling the system to inspect a user-supplied directory, identify supported source files, and preserve their relative paths. File-extension checks are used to restrict analysis to Python, JavaScript, TypeScript, and TSX files, while excluded directories and unsupported files are omitted from processing according to the implemented configuration.

Structured data exchanged between stages is represented using native Python dictionaries, lists, and JSON-compatible objects. These structures are used to store extracted-function metadata, generated descriptions, retrieved knowledge records, and vulnerability-analysis results. JSON serialization also supports the persistence of intermediate outputs and the validation of structured responses returned by the reasoning model.

Environment variables are used to manage sensitive and configurable values, particularly the OpenAI API key and model settings. The python-dotenv package loads these values from a local environment file, preventing credentials from being embedded directly in the source code. Configuration values such as supported extensions, knowledge-base paths, Chroma collection settings, retrieval parameters, and model identifiers are maintained separately from the main processing logic.

Logging and error-handling utilities are incorporated to record the status of the different pipeline stages and to identify failures such as unsupported files, parsing errors, empty functional descriptions, malformed model responses, or unsuccessful API calls. This is particularly important because the system performs multiple operations for each extracted function and must preserve sufficient diagnostic information when an individual stage fails.

The final audit output is generated locally through Python-based HTML construction or templating utilities. Structured findings are converted into sections for confirmed vulnerabilities, uncertain results, and functions classified as not vulnerable. Source paths, line ranges, CWE predictions, explanations, and remediation guidance are inserted into the report, while internal retrieval metadata and intermediate processing records remain outside the user-facing output.

The use of custom Python modules gives the implementation direct control over parsing, prompt construction, retrieval, response validation, and report assembly. It also allows individual components, such as the embedding model, reasoning model, parser, or vector store, to be modified independently without introducing an additional orchestration dependency.

3.3.2 Tree-sitter and Language Grammars

Tree-sitter is used as the language-aware parsing component of the proposed security auditor. Its role is to transform supported source files into structured syntax trees that can be traversed programmatically to identify relevant source-code constructs. Unlike line-based or regular-expression-based extraction, Tree-sitter provides access to the hierarchical structure defined by the

grammar of each programming language, allowing the system to locate complete classes, functions, and methods more reliably.

The current implementation supports Python, JavaScript, TypeScript, and TSX. A separate Tree-sitter grammar is used for each supported language so that the parser can interpret language-specific constructs correctly. This distinction is necessary because similar programming concepts may be represented differently across grammars. For example, a Python function is typically represented through a function-definition node, whereas JavaScript and TypeScript may distinguish among function declarations, function expressions, arrow functions, and method definitions.

During directory auditing, the system selects the appropriate grammar according to the file extension and parses the complete source file. The generated syntax tree is then traversed using language-specific extraction logic to locate classes, functions, and methods. Each extracted unit is stored together with its function or method name, programming language, source file path, starting line, and ending line. These location fields are preserved throughout the later stages of the pipeline so that generated findings can be associated with their original positions in the submitted project.

Tree-sitter is used only for structural source-code extraction in the current prototype. It does not perform vulnerability detection, semantic similarity computation, or security classification. It is also not used to construct control-flow graphs, data-flow graphs, call graphs, program slices, or intraprocedural relations. Likewise, the generated syntax trees are not converted into embeddings. Their purpose is to provide well-defined function-level analysis units and reliable source-location metadata for subsequent description generation, retrieval, vulnerability reasoning, and report generation.

The use of language-specific grammars therefore provides a consistent preprocessing layer across the supported languages while preserving the structural differences among them. This allows the system to accept a project directory containing multiple supported file types and convert its contents into traceable, function-level units without relying on manual snippet selection.

3.3.3 Knowledge-Base Source and Record Structure

The knowledge base used to ground the system's vulnerability analysis was derived from SecureCode v2.0. However, the original dataset was not used directly during system execution. Instead, it was processed, filtered, and reorganized into a project-specific knowledge base containing both vulnerable and secure code examples. The prepared knowledge base contains 764 security records. Each record contains a vulnerable source-code example together with a corresponding secure implementation and associated security metadata, including CWE information, functional description, category information, and other contextual fields.

The inclusion of both vulnerable and secure examples is important because the retrieval stage is not intended only to find code that resembles known vulnerabilities. It must also expose the reasoning model to examples of secure implementations so that the model can distinguish between similar functionality implemented safely and functionality containing an actual vulnerability mechanism. This design reduces the risk of treating every semantically similar function as vulnerable.

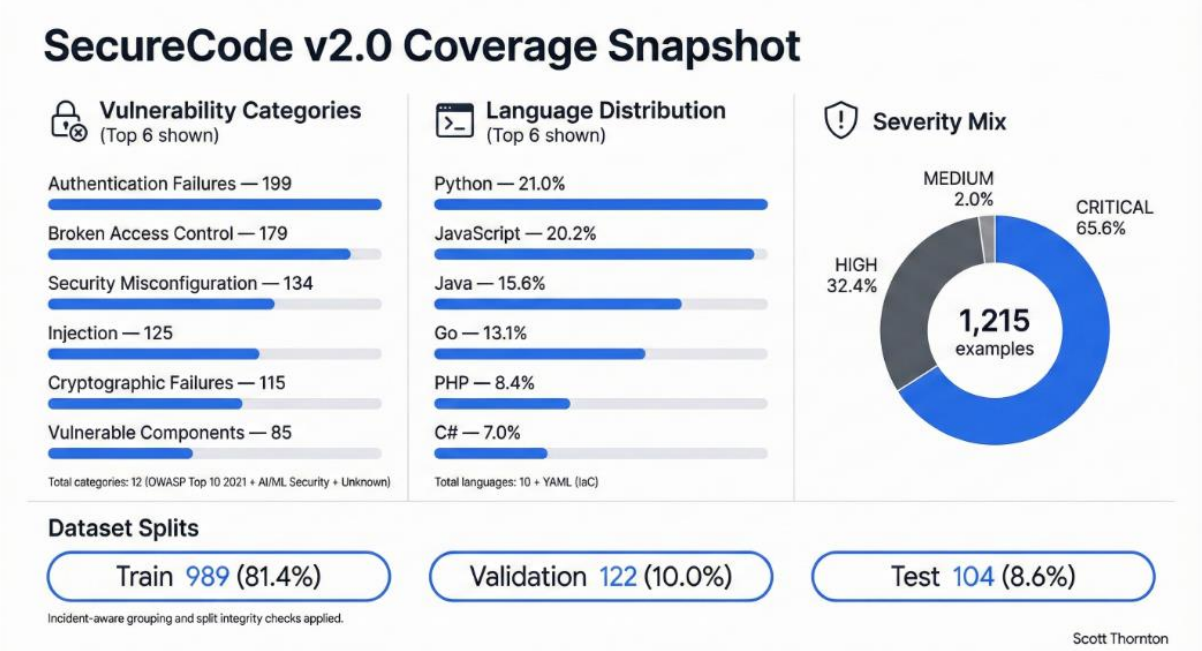

Figure 4 - Secure-Code v2.0 Coverage *[34]*.

Real-World Incident Grounding: A distinguishing characteristic of SecureCode v2.0 is that every example is rooted in a documented security incident with associated CVE identifiers. The dataset draws from actual security events spanning 2017–2025, such as the MoveIt Transfer SQL injection that impacted over 2,100 organizations with estimated damages of $92 billion. This CVE-grounded construction ensures that the knowledge base reflects real vulnerability patterns rather than synthetic or hypothetical examples. For this project, this grounding is critical: when the system retrieves a vulnerability example matching the input code, the LLM can reference the associated real-world incident to provide context-rich, evidence-backed audit reports.

3.3.3.1 Dataset Preparation and Enrichment

The original SecureCode-derived records were transformed into a normalized knowledge-base structure suitable for retrieval. During this preparation process, incomplete or unsuitable records were removed, the remaining examples were assigned stable source indices, and the fields needed by the retrieval and reasoning stages were retained.

Each final record contains information such as:

- a stable source_index;
- the programming language;
- a vulnerable source-code example;
- a corresponding secure implementation;
- a functional description;
- the associated CWE;
- category and subcategory information;
- attack-vector, impact, technique, and complexity information where available;
- CVE, CAPEC, OWASP, and real-world incident metadata where available.

The final record structure therefore contains more than raw source code. It combines the code example with semantic and security-oriented descriptions that can later support the reasoning model. Nevertheless, only one field is currently used for vector retrieval: the record's functional description for BGE and vulnerable code example for Unixcoder.

3.3.3.2 Vector Representation

The prepared knowledge base supports two alternative vector representations corresponding to the two retrieval configurations evaluated in this work. Rather than embedding all knowledge-base fields together, each configuration uses a single field selected according to its retrieval objective.

In the BGE-M3 configuration, only the functional_description field of each knowledge-base record is embedded using BAAI/bge-m3. This produces a functional-semantic representation intended to capture what the code performs at a higher level of abstraction. As a result, implementations with different syntax, identifiers, frameworks, or programming languages may still be retrieved as related when they express similar functionality.

In the UniXcoder configuration, only the vulnerable_code field is embedded using microsoft/unixcoder-base. This produces a direct code-oriented representation that preserves implementation-level characteristics such as API usage, operators, control structures, and other source-code patterns that may be relevant to vulnerability analysis. Unlike the BGE-M3 configuration, this approach does not require an intermediate natural-language description for retrieval.

The two representations therefore emphasize different forms of similarity: BGE-M3 prioritizes functional-semantic proximity, whereas UniXcoder prioritizes similarity in source-code representation. This distinction is central to the comparative design of the project, as it allows the effect of retrieval representation on downstream vulnerability reasoning and false-positive behavior to be evaluated while keeping the knowledge base and reasoning model consistent.

For both configurations, the resulting embeddings are stored in separate persistent ChromaDB collections. Each vector entry is linked to its original knowledge-base record through the corresponding source_index. Selected metadata fields are stored alongside the vector for identification and internal inspection, while the complete record remains available in final-knowledge-base.jsonl and is restored after retrieval for use by the reasoning stage.

3.3.3.3 Role in the System

The knowledge base serves as supporting context for the LLM-based vulnerability-reasoning stage. For each function or method extracted from the user-supplied project, the reasoning model receives the original source code together with the top three knowledge-base records retrieved by the selected embedding configuration. In the BGE-M3 configuration, the generated functional description is also available because it is produced as part of the semantic-retrieval process. In the UniXcoder configuration, retrieval operates directly on the source code and therefore does not require a generated functional description.

Using this information, GPT-5.1 produces a structured security assessment containing:

- a function-level verdict of vulnerable, not_vulnerable, or uncertain;
- one or more distinct vulnerability findings when the function is classified as vulnerable;
- the predicted CWE and CWE name for each identified weakness;
- the corresponding vulnerable source lines and code excerpt;
- a technical explanation of each vulnerability mechanism;
- a reasoning-based interpretation of the relevant input source, propagation, sensitive operation, and missing protection where these elements are visible;
- supporting knowledge-record identifiers where applicable; and
- remediation guidance for each confirmed weakness.

The retrieved knowledge records are treated as contextual evidence rather than as authoritative vulnerability labels. A high retrieval similarity does not imply that the analysed function is vulnerable, because functionally or structurally similar code may differ in security-critical implementation details. The reasoning model must therefore examine the target source code itself and determine whether a concrete vulnerability mechanism is actually present. This separation between retrieval and final assessment is important because the retrieval stage identifies potentially relevant evidence, whereas the reasoning stage performs the security judgement.

The retrieval candidates, similarity values, source indices, and internal knowledge-base records are preserved in intermediate processing files for debugging, traceability, and experimental analysis. These internal details are excluded from the final user-facing HTML report so that retrieved examples are not confused with confirmed findings in the submitted project. The final

report instead presents the validated reasoning output, including vulnerability classifications, affected source locations, explanations, and remediation recommendations.

3.3.4 BGE-M3 Embedding Model

The first retrieval configuration employs BAAI/bge-m3 to transform natural-language descriptions of source-code functionality into dense numerical vectors. These vectors are used to support functional-semantic retrieval from a persistent ChromaDB knowledge base [6]. In this configuration, the functional_description field of each knowledge-base record is embedded during ingestion, while GPT-5-mini generates a concise functional description for each extracted user function at audit time. The generated description is then embedded using the same BGE-M3 model to create the retrieval query.

BGE-M3 was selected because it is a general-purpose embedding model designed around multilinguality, multi-functionality, and multi-granularity. It supports more than 100 natural languages, can process inputs of varying length, and provides dense, sparse, and multi-vector retrieval representations. In the present implementation, only the dense embedding representation is used.

The purpose of this configuration is to retrieve records according to higher-level functional similarity rather than direct source-code similarity. For example, functions that authenticate users, execute database queries, access files, process redirect targets, or invoke operating-system commands may be represented as semantically related even when they differ in syntax, framework, identifier naming, or programming language.

This abstraction can reduce the influence of implementation-specific differences and support retrieval across syntactically diverse code. However, the resulting similarity reflects functional proximity rather than vulnerability equivalence. Security-critical implementation details may not always be fully preserved in a short natural-language description. For this reason, the original target source code remains part of the downstream GPT-5.1 reasoning input, and the retrieved records are treated only as supporting evidence.

Within the comparative design of this project, BGE-M3 therefore represents the functional-semantic retrieval strategy. Its performance is evaluated against a second configuration based on direct source-code representation using UniXcoder.

3.3.5 UnixCoder Embedding Model

The second retrieval configuration employs microsoft/unixcoder-base as a code-oriented embedding model for representing source code directly. Unlike the BGE-M3 configuration, UniXcoder does not require the source code to be converted into a natural-language functional description before retrieval. Instead, the target function and the vulnerable source-code examples stored in the knowledge base are encoded directly into dense vector representations [7].

During knowledge-base ingestion, the vulnerable_code field of each of the 764 prepared records is embedded using UniXcoder and stored in a dedicated persistent ChromaDB collection. At audit time, Tree-sitter extracts the complete source code of a function or method from the submitted project, and the extracted code is embedded using the same UniXcoder configuration. ChromaDB then retrieves the three nearest stored code representations, after which the corresponding complete knowledge-base records are restored through their source_index values and supplied to GPT-5.1 for vulnerability reasoning.

UniXcoder was selected because it was pretrained specifically for code representation and supports code-related tasks such as code search and code-to-code representation [7]. Its use allows the retrieval process to preserve implementation-level information contained directly in the source code, including API calls, operators, control structures, and coding patterns that may be relevant to vulnerability behavior.

The UniXcoder configuration therefore differs conceptually from the BGE-M3 approach. BGE-M3 compares natural-language descriptions of what functions do, whereas UniXcoder compares source-code representations of how those functions are implemented. This distinction is particularly relevant to vulnerability analysis because small implementation differences may determine whether otherwise similar functionality is secure or vulnerable.

As with BGE-M3, vector similarity is used only to rank candidate knowledge records and is not interpreted as a vulnerability probability or final classification. The retrieved records are passed to GPT-5.1 as contextual evidence, while the final security judgement is based on examination of the target function itself. Within the comparative design of this project, UniXcoder therefore represents the code-oriented retrieval strategy, allowing its effect on vulnerability detection and false-positive control to be compared directly with the BGE-M3 functional-semantic configuration.

3.3.6 ChromaDB

ChromaDB is employed as the persistent vector database responsible for storing and retrieving the dense representations of the prepared security knowledge base. It provides the retrieval layer that connects the functional description of an analyzed function with semantically related vulnerable and secure examples. The database stores the embeddings generated by BGE-M3 together with record identifiers and selected metadata, thereby enabling efficient similarity-based access to the 764 records contained in the knowledge base.

The implementation uses a locally persisted ChromaDB collection rather than an external database service. This configuration is appropriate for the current prototype because the knowledge base is relatively compact and can be indexed and queried within the local execution environment. Persistence also separates the knowledge-base construction phase from the auditing phase, since the vector collection does not need to be reconstructed whenever a new source-code project is analyzed.

3.3.6.1 Vector Collection Configuration

The system uses persistent ChromaDB collections to store the vector representations produced by the two retrieval configurations. Because BGE-M3 and UniXcoder generate embeddings from different input representations and occupy different vector spaces, each configuration uses a separate collection rather than sharing a single index.

In the BGE-M3 configuration, each of the 764 knowledge-base records is represented by an embedding of its functional_description field. In the UniXcoder configuration, each record is represented by an embedding of its vulnerable_code field. The corresponding collections therefore contain the same number of indexed knowledge-base records but differ in the information used to construct their vector representations.

Each ChromaDB entry is linked to the original record through the record's source_index, which provides a stable association between the stored vector and the corresponding entry in final-knowledge-base.jsonl. This allows the retrieval module to recover the complete security record after ChromaDB returns a nearest-neighbour match.

Selected metadata fields are stored alongside the vector entries to support identification and internal inspection. In the UniXcoder configuration, the stored metadata includes the source index, CWE, programming language, category, and subcategory. Additional security information remains in the complete JSONL record and is restored only after retrieval. Candidate selection in the evaluated configurations is based on vector-space proximity rather than metadata filtering.

3.3.6.2 Similarity Search

The collection is configured to use cosine distance for nearest-neighbour retrieval. This measure compares the orientation of the query vector with the stored knowledge-base vectors. Functional descriptions with similar meanings are expected to be positioned closer together in the embedding space, whereas descriptions representing different behaviours are expected to be farther apart.

At audit time, the functional description generated for each submitted function is embedded using the same BGE-M3 model employed during knowledge-base ingestion. ChromaDB compares the resulting query vector with the stored vectors and returns the three nearest records.

The reported similarity values represent the semantic proximity between the functional description of the analysed function and the descriptions stored in the knowledge base. They must not be interpreted as vulnerability probabilities, model confidence scores, or final security classifications. A secure implementation and a vulnerable implementation may perform the same functional operation and therefore receive similar retrieval scores. The final determination is produced by the subsequent reasoning stage, which examines the submitted source code and the complete retrieved records.

3.3.6.3 Persistent Storage

Both retrieval configurations use cosine-based nearest-neighbour search to rank knowledge-base records according to the representation of the analysed function. The interpretation of the resulting proximity depends on the embedding configuration being used.

In the BGE-M3 configuration, GPT-5-mini first generates a concise functional description of the extracted function. BGE-M3 encodes this description into a dense query vector, which is compared with the stored functional-description embeddings. The resulting ranking therefore reflects similarity in higher-level functional semantics.

In the UniXcoder configuration, the extracted source code itself is encoded directly using microsoft/unixcoder-base and compared with the stored embeddings of the vulnerable_code fields. In this case, the ranking reflects proximity in code-oriented representation space rather than similarity between generated natural-language descriptions.

For both configurations, ChromaDB returns the three highest-ranked knowledge-base records. The retrieval scores are used only to rank candidate evidence and must not be interpreted as vulnerability probabilities, confidence scores, or final security classifications. A highly ranked record indicates representational similarity under the selected embedding model, but the final vulnerability decision is produced separately by GPT-5.1 after examination of the submitted source code and the complete retrieved records.

3.3.6.4 Retrieval and Full-Record Hydration

The vector collections are stored persistently in local ChromaDB directories so that the prepared knowledge-base embeddings can be reused across multiple auditing sessions. Once the 764 knowledge-base records have been embedded and indexed for a particular retrieval configuration, subsequent project scans can load the existing collection directly rather than reconstructing the complete vector index.

Knowledge-base preparation is therefore separated from online auditing. For the BGE-M3 configuration, the offline ingestion stage embeds the functional_description field of each knowledge-base record and stores the resulting vectors in its dedicated ChromaDB collection. For the UniXcoder configuration, the ingestion stage instead embeds the vulnerable_code field and stores those representations in a separate persistent collection.

During auditing, only the submitted functions require new query embeddings. BGE-M3 embeds generated functional descriptions, whereas UniXcoder embeds the extracted source code directly. This separation reduces redundant computation and ensures that repeated audits using the same configuration operate over an unchanged indexed knowledge base.

Re-ingestion is required when a change affects the stored representation, such as modification of the knowledge base, embedded field, embedding model, or vector-collection configuration.

A hydrated record may include:

- vulnerable source code;
- corresponding secure source code;
- functional description;
- CWE information;
- category and subcategory;
- severity, complexity, and technique information;
- attack-vector and impact information;
- CVE, CAPEC, and OWASP metadata where available;
- real-world incident information where available.

The hydrated records are then written to the retrieval output and supplied to GPT-5.1 together with the original target function. This design keeps ChromaDB focused on efficient vector retrieval while the JSONL knowledge base preserves the richer security context required for reasoning and analysis.

3.3.6.5 Role in the System

ChromaDB functions as the vector retrieval layer of the RAG-based auditing pipeline. It does not perform structural parsing, vulnerability reasoning, CWE classification, or final security assessment. Its role is to identify the knowledge-base records that are nearest to the analysed function under the selected retrieval representation.

The meaning of retrieval relevance differs between the two configurations. BGE-M3 retrieves records according to functional-semantic similarity between generated descriptions, whereas UniXcoder retrieves records according to direct source-code representation. In both cases, the retrieved records are treated as contextual evidence rather than as authoritative labels.

The reasoning model must therefore examine the submitted source code and determine whether the security mechanism represented by the retrieved evidence is actually present. A retrieved record may be highly similar yet not correspond to a vulnerability in the target function, particularly when the target contains an effective protection mechanism. This separation between retrieval and vulnerability assessment ensures that vector similarity guides evidence selection without directly determining the final verdict.

3.3.7 Large Language Models

Large language models are used in different roles across the two retrieval configurations of the implemented security-auditing pipeline. GPT-5.1 is the common downstream vulnerability-reasoning model used in both configurations. GPT-5-mini is additionally used in the BGE-M3 configuration to generate concise functional descriptions of extracted source-code units before

semantic retrieval. In the UniXcoder configuration, this intermediate description-generation stage is not required because retrieval operates directly on source-code embeddings.

The role of the language models is separated from structural parsing and vector retrieval. Tree-sitter is responsible for locating functions, methods, classes, file paths, and line ranges. BGE-M3 and UniXcoder provide alternative vector representations for retrieval, while ChromaDB performs nearest-neighbour search over the corresponding persistent collections. GPT-5-mini supports semantic abstraction only in the BGE-M3 configuration, whereas GPT-5.1 performs the final security assessment using the submitted source code and the retrieved knowledge-base records.

The resulting RAG architecture therefore combines structural source-code extraction, alternative retrieval representations, external security knowledge, and LLM-based reasoning. Retrieved records provide supporting evidence rather than authoritative vulnerability labels. The final verdict is produced only after GPT-5.1 examines the target source code and determines whether one or more concrete vulnerability mechanisms are actually present.

3.3.7.1 Functional-Description Generation

Functional-description generation is used only in the BGE-M3 retrieval configuration. For each function or method extracted by Tree-sitter, GPT-5-mini generates a concise natural-language description of the unit's functional behavior. The purpose of this description is to summarize what the code performs without carrying out vulnerability classification, CWE prediction, or remediation analysis.

The generated description serves as the semantic query for the BGE-M3 retrieval stage. It is embedded using BAAI/bge-m3 and compared with the functional_description embeddings of the 764 knowledge-base records stored in the corresponding ChromaDB collection. This representation is intended to reduce dependence on source-code syntax, identifier naming, formatting, framework-specific conventions, and programming-language differences.

The description-generation prompt includes the programming language, function or method name, extracted source code, and instructions to summarize the function in one concise sentence without performing security analysis, reproducing the source code, or returning markdown formatting.

The generated description is preserved with the extracted function data and is subsequently used by the BGE-M3 retrieval module. If description generation fails or produces an unusable response, the failure is recorded in the corresponding error output so that the affected function is not silently omitted from the processing workflow.

The UniXcoder configuration does not use this stage because its retrieval query is constructed directly from the extracted source code.

3.3.7.2 Vulnerability-Reasoning Prompt Structure

The vulnerability-reasoning stage is implemented through the OpenAI Responses API using GPT-5.1. For each analysed function, the model receives a structured input containing the original function source code, source file path, programming language, function or method name, start and end line numbers, and the top three hydrated knowledge-base records retrieved by the selected ChromaDB configuration.

In the BGE-M3 configuration, the generated functional description is also included because it forms part of the semantic-retrieval process. In the UniXcoder configuration, the reasoning stage does not depend on a generated functional description because retrieval is performed directly using source-code embeddings.

Each retrieved knowledge-base record may provide vulnerable source code, corresponding secure code, CWE information, functional description, category information, attack-related metadata, and other security context. These records are presented as supporting evidence rather than as ground-truth labels for the submitted function.

The reasoning prompt instructs GPT-5.1 to analyse the submitted source code itself and to return one valid structured JSON object. The function-level output contains fields such as:

- analysis_status;
- verdict;
- is_vulnerable;
- reason;
- vulnerabilities;
- matched_knowledge.

The vulnerabilities field is a list that may contain zero, one, or multiple distinct vulnerability findings. Each finding may contain:

- cwe;
- cwe_name;
- vulnerable_lines;
- vulnerable_code_excerpt;
- reason;
- data_flow;
- evidence_source_indices;

- recommended_fix.

The function-level verdict is constrained to three possible values:

1. vulnerable;
2. not_vulnerable;
3. uncertain.

A function classified as vulnerable may contain one or more independent weaknesses. A not_vulnerable or uncertain result contains an empty vulnerabilities list.

The prompt further instructs the model to analyse the entire function rather than stopping after the first weakness. Distinct findings are reported only when they represent independent vulnerability mechanisms, while multiple CWE labels describing the same underlying weakness are not treated as separate vulnerabilities.

3.3.7.3 Reasoning and Classification

GPT-5.1 is responsible for determining whether one or more concrete vulnerability mechanisms are visible in the supplied function. The reasoning model examines the target source code and compares it with the retrieved vulnerable and secure examples. It is explicitly instructed not to classify a function as vulnerable solely because a similar vulnerable example was retrieved.

The reasoning process considers factors such as:

- the origin of potentially untrusted input;
- how that input propagates through the function;
- the sensitive operation or security-relevant sink;
- the presence or absence of protections such as validation, parameterization, encoding, authorization, containment, or sanitization;
- whether the retrieved vulnerability mechanism is actually present in the submitted source code;
- whether multiple independent weaknesses occur within the same function;
- whether missing external context materially prevents a reliable judgement.

When sufficient evidence is visible, the model returns either vulnerable or not_vulnerable. The uncertain category is reserved for cases in which unavailable context is genuinely necessary to determine the security outcome.

The reasoning model is also instructed not to reject an existing protection merely because a stronger alternative could be used. A vulnerability finding must instead be supported by a concrete weakness mechanism visible in the submitted source code.

The data_flow field provides an LLM-inferred interpretation of the relevant source-to-sink relationship. It may describe the input source, propagation path, sensitive sink, and missing protection associated with a finding. This field is not generated by Tree-sitter and does not represent the output of a formal static taint-analysis engine. It is therefore treated as reasoning-based explanatory information rather than formally verified program analysis.

3.3.7.4 CWE Identification and Remediation

For each distinct vulnerability identified within a function, GPT-5.1 predicts one primary CWE identifier and CWE name corresponding to the observed weakness mechanism. Retrieved knowledge-base records may support this classification, but the model is required to base the final decision on the actual submitted source code rather than assuming that the CWE associated with a retrieved record also applies to the target function.

The reasoning output also identifies the relevant source lines and provides a concise code excerpt corresponding to the unsafe operation or vulnerability mechanism. Because one function may contain several independent vulnerabilities, each finding maintains its own CWE, affected lines, technical explanation, evidence references, and remediation recommendation.

Remediation guidance is generated specifically for the identified weakness. Depending on the vulnerability mechanism, this may include parameterized database queries, safer command execution, path validation and containment, output encoding, authorization checks, secure secret handling, safer parsing or serialization mechanisms, or other protections appropriate to the identified CWE.

For uncertain results, no confirmed vulnerability object is produced. Instead, the function-level explanation describes the missing context required for a reliable assessment.

3.3.7.5 Output Processing and Report Generation

The structured reasoning results are written to vulnerability_findings.jsonl, where each record preserves the complete analysis result for one function. Processing failures are written to reasoning_errors.jsonl, while aggregate execution statistics are stored in reasoning_summary.json.

A single function-level record may contain zero, one, or multiple vulnerability objects. The summary therefore distinguishes between the number of vulnerable functions and the total number of distinct vulnerability findings.

The detailed findings file retains internal information used for debugging, evaluation, and traceability, including retrieval candidates and evidence identifiers. These internal records are not exposed directly to the end user.

A separate report-generation component transforms the structured reasoning output into a user-facing HTML security audit report containing:

- a summary of analysed functions, vulnerable functions, distinct vulnerability findings, uncertain results, and non-vulnerable functions;
- individual detailed cards for confirmed vulnerabilities;
- separate presentation of multiple vulnerabilities identified within the same function;
- concise explanations for uncertain findings;
- a compact table of functions classified as not vulnerable;
- analysis failures, when present.

Each confirmed vulnerability is presented independently with its CWE, affected lines, technical explanation, vulnerable-code excerpt, and recommended remediation. Retrieved examples, vector similarity scores, source indices, and internal reasoning metadata are excluded from the final report so that supporting retrieval context is not confused with confirmed findings in the submitted project.

3.3.7.6 Role in The System

The language-model components perform two clearly separated functions within the implemented architecture. GPT-5-mini provides functional abstraction for the BGE-M3 semantic-retrieval configuration, while GPT-5.1 serves as the common downstream vulnerability-reasoning model for both the BGE-M3 and UniXcoder configurations.

This separation allows the project to compare two alternative retrieval representations without changing the final reasoning role. BGE-M3 supplies evidence through functional-semantic similarity, whereas UniXcoder supplies evidence through direct source-code similarity. In both cases, GPT-5.1 determines whether the retrieved evidence corresponds to a concrete vulnerability mechanism in the target function.

The complete technical stack therefore consists of Python for pipeline coordination, Tree-sitter for structural source-code extraction, GPT-5-mini for functional-description generation in the BGE-M3 configuration, BGE-M3 and UniXcoder for alternative retrieval representations, ChromaDB for persistent vector retrieval, GPT-5.1 for structured multi-vulnerability reasoning, and a dedicated Python reporting component for generating the final HTML and JSON security audit outputs.

3.4 Project Setup Procedure

The project environment was established as a local Python workspace organized into separate directories for the prepared knowledge base, source-code processing outputs, persistent vector storage, and implementation modules. The prepared final-knowledge-base.jsonl file, containing

764 security records with vulnerable and corresponding secure code examples, was used as the source for the knowledge-base ingestion stage. Persistent ChromaDB collections were constructed for the two retrieval configurations so that the indexed knowledge base could be reused across multiple auditing sessions without regenerating all embeddings for each submitted project.

The required Python dependencies were installed to support structured source-code parsing, data handling, embedding generation, vector retrieval, and communication with the OpenAI API. These dependencies included Tree-sitter and its language bindings, the libraries required for BGE-M3 and UniXcoder embedding inference, PyTorch, ChromaDB, and the OpenAI Python SDK. The OpenAI API key was configured through environment variables rather than being embedded directly in the source code, thereby reducing the risk of accidental credential disclosure.

The setup procedure was divided into an offline preparation stage and an online auditing stage. During the offline stage, the prepared 764-record JSONL knowledge base was loaded and indexed according to the selected retrieval configuration. In the BGE-M3 configuration, the functional_description field of each knowledge-base record was embedded using BAAI/bge-m3 and stored in a dedicated persistent ChromaDB collection. In the UniXcoder configuration, the vulnerable_code field of each record was embedded directly using microsoft/unixcoder-base and stored in a separate ChromaDB collection. In both cases, the source_index associated with each vector was preserved so that the complete knowledge-base record could later be restored after retrieval.

During the online auditing stage, the user supplies a source-code directory, which is recursively processed using Tree-sitter to extract functions and methods from Python, JavaScript, TypeScript, and TSX files while preserving their file paths and source-line ranges. The subsequent retrieval process depends on the selected configuration. In the BGE-M3 configuration, GPT-5-mini generates a concise functional description for each extracted unit, which is then embedded using BGE-M3 and compared with the stored functional-description vectors. In the UniXcoder configuration, the extracted source code is embedded directly and compared with the stored vulnerable-code representations. In both configurations, ChromaDB returns the three highest-ranked knowledge-base records.

The retrieved entries are subsequently hydrated from final-knowledge-base.jsonl using their source_index values. The complete records are then supplied, together with the original analysed function and its source-location information, to GPT-5.1 through the OpenAI Responses API. In the BGE-M3 configuration, the generated functional description is additionally available to the reasoning stage.

The reasoning stage produces a structured function-level verdict of vulnerable, not_vulnerable, or uncertain. When a function is classified as vulnerable, the result may contain one or more distinct vulnerability findings, each including a predicted CWE and CWE name, vulnerable source lines, a relevant code excerpt, technical explanation, reasoning-based data-flow information where

applicable, supporting evidence identifiers, and remediation guidance. This structure allows multiple independent weaknesses occurring within the same function to be represented separately.

A dedicated report-generation module then processes the structured reasoning results and produces user-facing HTML and JSON security audit reports. The report distinguishes the number of vulnerable functions from the total number of distinct vulnerability findings and presents each confirmed weakness with its associated source location, CWE classification, explanation, and recommended remediation. Internal retrieval information such as vector similarity scores, source indices, and retrieved examples is retained for internal processing and evaluation but is not exposed as part of the final user-facing findings.

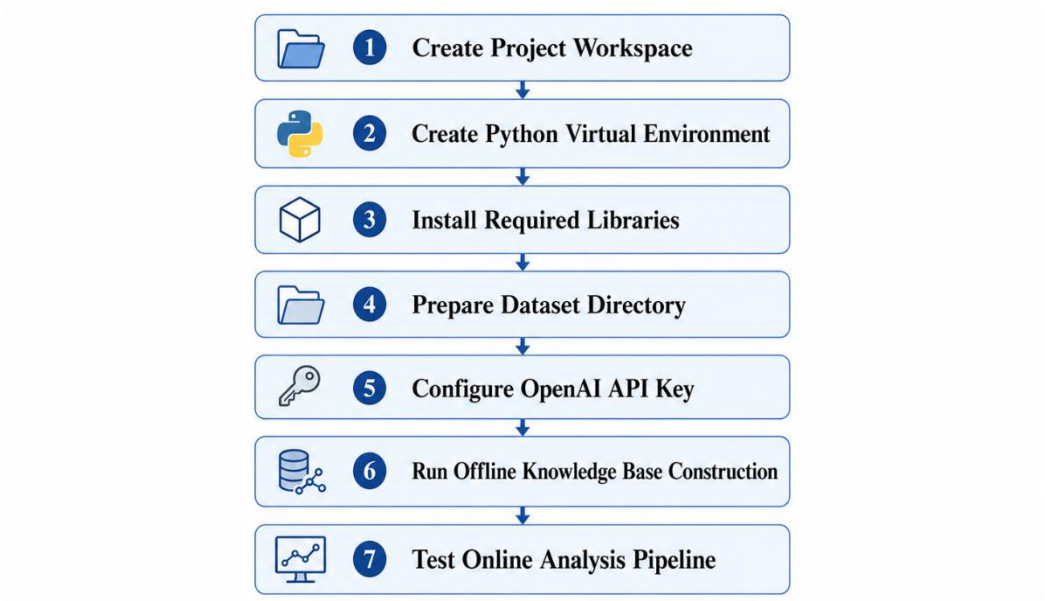

Figure 5 - Project Setup Procedure

Figure 5 illustrates the principal steps involved in preparing the development environment and executing the prototype. The setup begins with the local Python workspace and required dependencies, proceeds through offline knowledge-base indexing, and then supports online directory-based auditing through source-code extraction, retrieval, LLM-based vulnerability reasoning, and final report generation.

3.5 Conclusion

This chapter presented the development environment and principal technologies used to implement the RAG-based security auditor. The system employs Python and Tree-sitter for pipeline coordination and structural source-code extraction, respectively, while ChromaDB provides persistent vector storage and nearest-neighbour retrieval. Two alternative retrieval configurations are supported: BGE-M3 performs functional-semantic retrieval over generated functional descriptions, whereas UniXcoder performs direct code-oriented retrieval over source-code representations. GPT-5-mini is used only in the BGE-M3 configuration to generate functional

descriptions, while GPT-5.1 serves as the common downstream model for structured vulnerability reasoning.

The chapter also described the prepared 764-record knowledge base, persistent vector-storage strategy, language-model services, and offline and online setup procedures. Together, these components establish the technical environment required for the implementation and experimental comparison of the two retrieval representations described in the following chapter.

Chapter 4: Practical Implementation

4.1 Introduction

This chapter presents the practical implementation of the proposed retrieval-augmented generation (RAG) security auditor. The prototype accepts a user-specified source-code directory, extracts function-level program units, retrieves relevant security knowledge, performs structured vulnerability reasoning, and generates user-facing HTML and JSON audit reports. The implementation is modular so that preprocessing, retrieval, reasoning, and reporting can be inspected and executed independently.

The implemented work contains two alternative retrieval configurations that share the same prepared knowledge base and downstream reasoning stage. The BGE-M3 configuration represents each function through a GPT-5-mini-generated functional description and performs functional-semantic retrieval. The UniXcoder configuration embeds source code directly using microsoft/unixcoder-base and performs code-oriented retrieval. In both configurations, the three highest-ranked knowledge-base records are hydrated from the complete JSONL source and supplied to GPT-5.1 as supporting evidence for the final security assessment.

The final command-line prototype uses the UniXcoder retrieval configuration for end-to-end directory auditing. The BGE-M3 configuration is retained as the alternative RAG configuration used in the comparative evaluation. This chapter focuses on implementation and operational behavior; quantitative benchmark results are presented separately in the evaluation chapter.

4.2 Implemented System Architecture

The system follows an Ingestion/Retrieval architecture. During ingestion, the 764-record security knowledge base is embedded once and persisted in ChromaDB. During online auditing, the submitted directory is parsed with Tree-sitter, function-level units are extracted, the selected retrieval configuration is applied, and the resulting evidence is passed to GPT-5.1 for structured reasoning. The final results are then transformed into an HTML security audit report and a machine-readable JSON report.

The two retrieval configurations differ only in the representation used to search the knowledge base. BGE-M3 retrieves through natural-language functional descriptions, whereas UniXcoder retrieves through direct source-code representations. Retrieval similarity is used only to rank evidence and is never treated as a vulnerability probability or as the final security verdict.

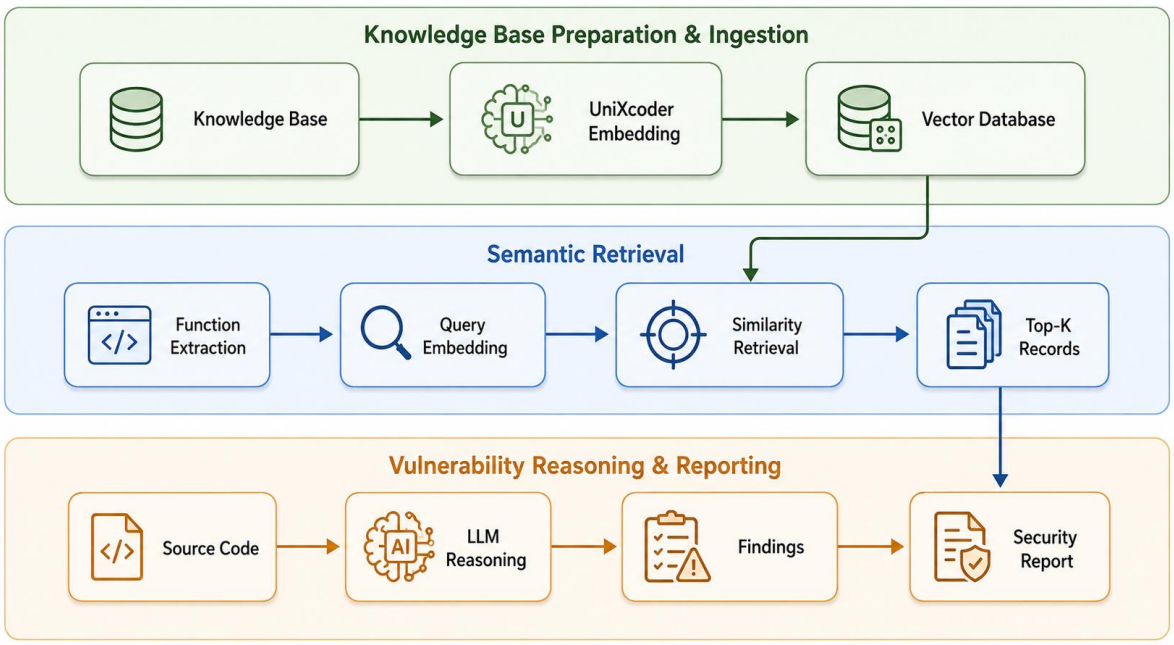

Figure 6 - UniXcoder-based implementation workflow of the RAG security auditor.

Figure 6 summarizes the operational UniXcoder workflow. The prepared knowledge base is embedded and persisted in a vector database. At audit time, Tree-sitter-extracted source-code functions are encoded as query vectors and compared with the stored knowledge representations. The top-ranked records are then combined with the original source code for GPT-5.1 reasoning, after which structured findings are converted into the final security report.

Table 3 - Principal implementation modules and outputs.

Module	Primary responsibility	Principal output
ingestion_unixcoder_cli.py	Embeds vulnerable_code records with UniXcoder and builds the persistent vector collection.	Persistent ChromaDB collection
users_code_preprocessing_cli.py	Parses the submitted directory and extracts classes, functions, and methods with locations.	method_data.csv, class_data.csv
retrieval_unixcoder_cli.py	Embeds target source code, retrieves the top three records, and hydrates complete JSONL entries.	retrieval_results.jsonl
vulnerability_reasoning_cli.py	Uses GPT-5.1 to produce structured function-level verdicts and zero, one, or multiple vulnerability findings.	vulnerability_findings.jsonl, reasoning_errors.jsonl, reasoning_summary.json
report_generation_cli.py	Transforms structured reasoning outputs into user-facing reports.	security_report.html, security_report.json

rag_auditor.py	Coordinates preprocessing, retrieval, reasoning, and reporting through one command-line entry point.	End-to-end project audit

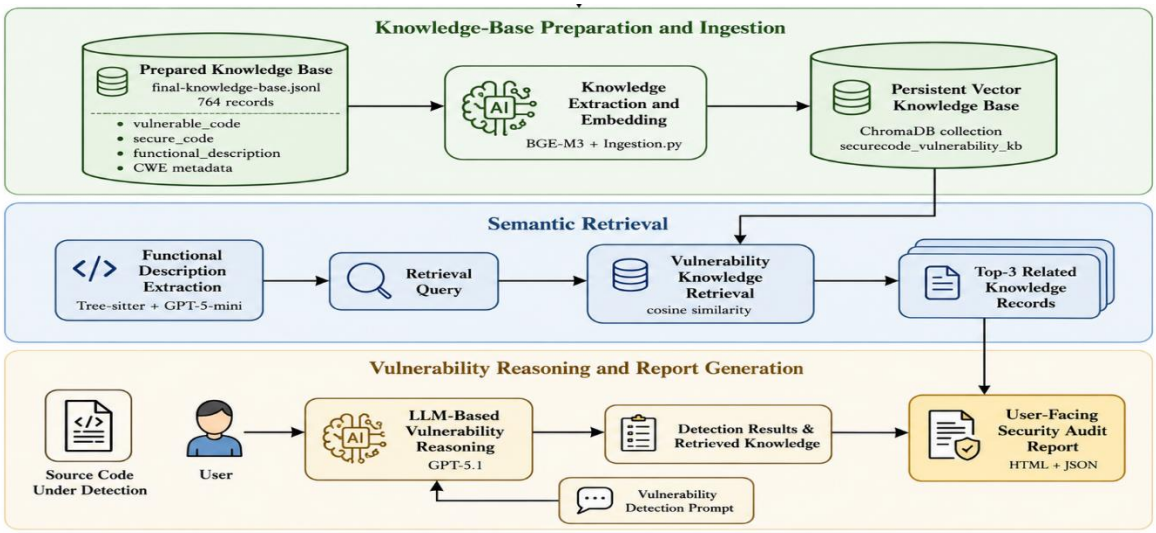

Figure 7 - RAG-based implementation workflow of the RAG security auditor.

The architecture is implemented through five principal modules. The ingestion module constructs the persistent vector knowledge base. The preprocessing module parses a source-code directory and generates function-level descriptions. The retrieval module performs dense similarity search and reconstructs complete knowledge records. The reasoning module generates structured security findings. The report-generation module produces concise user-facing outputs while retaining internal retrieval details in machine-readable files for debugging and evaluation.

Table 4 - Principal implementation modules and outputs.

Module	Primary responsibility	Principal output
ingestion.py	Embed prepared knowledge-base descriptions and persist the vector collection.	Persistent ChromaDB collection
users_code_preprocessing.py	Parse supported source files, extract functions and methods, and generate functional descriptions.	method_data.csv, class_data.csv, description_errors.csv
retrieval.py	Retrieve the three nearest records and hydrate complete JSONL entries.	retrieval_results.jsonl, retrieval_skipped.csv
vulnerability_reasoning.py	Assess each extracted method with GPT-5.1 and produce structured findings.	vulnerability_findings.jsonl, reasoning_errors.jsonl, reasoning_summary.json

Module	Primary responsibility	Principal output
report_generation.py	Transform reasoning outputs into concise user-facing reports.	security_report.html, security_report.json

4.3 Knowledge-Base Ingestion

The offline ingestion stage begins from final-knowledge-base.jsonl, a prepared file containing 764 security records. Each record is associated with a stable source_index and contains a vulnerable code example, a corresponding secure implementation, a functional description, CWE information, and additional security metadata. The complete record remains in JSONL form so that retrieval can use a lightweight vector entry while the reasoning stage can later recover the richer context.

4.3.1 Input Record Handling

The ingestion procedure reads the JSONL file as a sequence of structured records and validates the fields required by the selected embedding configuration. Every vector entry is associated with the record's source_index, which provides a deterministic link between the vector store and the original JSONL record. This identifier is later used during full-record hydration.

The two configurations use different fields for vector construction. BGE-M3 embeds the functional_description field. UniXcoder embeds the vulnerable_code field. Other fields, including secure_code, CWE, category, subcategory, attack-related information, and incident metadata, remain available in the complete JSONL record but are not concatenated into the retrieval vector.

4.3.2 Alternative Embedding Strategies

In the BGE-M3 configuration, each functional description is converted into a dense vector using BAAI/bge-m3. This representation emphasizes what the example does and reduces dependence on syntax, identifier naming, formatting, and programming-language differences. At audit time, GPT-5-mini generates the corresponding functional description for the submitted function before BGE-M3 retrieval.

In the UniXcoder configuration, the vulnerable_code field is embedded directly using microsoft/unixcoder-base. The implementation uses the model's first-token representation and L2 normalization to produce the stored code vector. This approach preserves implementation-level information without requiring an intermediate natural-language description and forms the retrieval configuration used by the final command-line auditing prototype.

4.3.3 Persistent ChromaDB Collections

Separate persistent ChromaDB collections are maintained for the two embedding spaces. This separation is necessary because BGE-M3 and UniXcoder produce representations with different

semantics and must not be mixed within the same nearest-neighbor index. Persistence allows the 764 knowledge records to be embedded once and reused for subsequent audits.

Table 5 - Implemented knowledge-base retrieval configurations.

Configuration item	BGE-M3 configuration	UniXcoder configuration
Knowledge-base source	final-knowledge-base.jsonl	final-knowledge-base.jsonl
Number of records	764	764
Embedding model	BAAI/bge-m3	microsoft/unixcoder-base
Embedded field	functional_description	vulnerable_code
Query representation	GPT-5-mini functional description	Raw extracted source code
Vector database	ChromaDB	ChromaDB
Retrieval depth	Top 3	Top 3
Record link	source_index	source_index

4.4 User-Code Directory Preprocessing

The online auditing process starts with directory preprocessing. The user supplies the path of a source-code repository, after which the preprocessing module recursively identifies supported files and parses them with Tree-sitter. The current implementation supports Python, JavaScript, TypeScript, and TSX.

4.4.1 Tree-sitter Parsing and Unit Extraction

For each supported file, the appropriate Tree-sitter grammar is selected and the complete file is parsed. The implementation extracts classes, functions, and methods together with the programming language, file path, class association where applicable, function name, start line, end line, original source code, and discovered references. The resulting function and method records are written to method_data.csv and form the analysis units used by retrieval and reasoning.

Tree-sitter is used only for structural extraction and source-location preservation. It does not perform vulnerability classification, taint analysis, control-flow analysis, or embedding generation. This separation keeps the preprocessing stage deterministic and allows the subsequent retrieval and LLM components to operate on well-defined function-level units.

4.4.2 Retrieval-Specific Preprocessing

The preprocessing requirements differ between the two retrieval configurations. In the BGE-M3 configuration, GPT-5-mini generates a concise functional description for each extracted unit, and

this text becomes the retrieval query. In the UniXcoder configuration, no description-generation stage is required for retrieval; the extracted source_code field itself becomes the query input.

The final command-line implementation therefore avoids an additional LLM request before retrieval when UniXcoder is selected. The original source code and location metadata are preserved regardless of the retrieval configuration because GPT-5.1 must inspect the submitted implementation when making the final security judgement.

Table 5 - Principal preprocessing output fields.

Field	Purpose
method_id	Stable identifier for the extracted analysis unit.
unit_type	Distinguishes functions and methods.
file_path	Preserves the original source file location.
language	Identifies the programming language and parser grammar.
class_name	Stores the enclosing class where applicable.
name	Function or method name.
start_line / end_line	Preserves the source location used in findings and reports.
source_code	Stores the original extracted implementation for retrieval and reasoning.
references	Stores references discovered during structural preprocessing.

4.5 Retrieval Pipeline

The retrieval stage converts each extracted function into a query representation and searches the corresponding persistent ChromaDB collection. The BGE-M3 and UniXcoder configurations use the same overall retrieval logic but differ in the representation supplied to the vector search.

4.5.1 Query Embedding and Top-k Search

In the BGE-M3 configuration, the GPT-5-mini-generated functional description is embedded with BGE-M3 and compared with the stored functional-description vectors. In the UniXcoder configuration, the original function source code is embedded directly and compared with stored vulnerable-code representations. The current retrieval depth is Top-K = 3.

The returned distance or similarity values are used only for ranking the candidate records. They are not treated as confidence scores, vulnerability probabilities, or final labels. A highly ranked example can be only partially relevant, and the subsequent reasoning model is required to verify whether the demonstrated weakness mechanism actually appears in the submitted code.

4.5.2 Full-Record Hydration

ChromaDB stores the vector entry and lightweight identifying information rather than duplicating every security field. After the three nearest entries are returned, the retrieval module uses each source_index to load the complete corresponding record from final-knowledge-base.jsonl. This hydration step restores vulnerable_code, secure_code, functional_description, CWE information, category and subcategory information, and the additional security metadata retained in the prepared knowledge base.

The hydrated records are written to the retrieval output and then supplied to GPT-5.1. This design keeps vector storage compact while preserving the full knowledge record for security reasoning.

4.6 LLM-Based Vulnerability Reasoning

The vulnerability-reasoning stage is implemented in vulnerability_reasoning_cli.py and uses GPT-5.1 through the OpenAI Responses API. One reasoning request is constructed for each extracted function. The request contains the original source code, project location metadata, and the top three hydrated knowledge-base records. In the BGE-M3 configuration, the generated functional description is additionally available to the model.

4.6.1 Structured Reasoning Input

Retrieved knowledge is explicitly treated as supporting evidence rather than as ground truth. The prompt instructs GPT-5.1 to analyse the submitted source code itself, compare it with the retrieved vulnerable and secure examples, and use the retrieved records only when they are relevant to the observed implementation. Weak or unrelated retrieval does not prevent direct source-code analysis.

The prompt also requires analysis of the complete function and permits zero, one, or multiple independent vulnerabilities. Separate findings are created only for distinct vulnerability mechanisms; multiple CWE labels describing the same underlying weakness are not treated as separate findings.

4.6.2 Verdict and Multi-Vulnerability Output

The function-level verdict is constrained to vulnerable, not_vulnerable, or uncertain. A vulnerable function contains a vulnerabilities list with one or more structured findings. A not_vulnerable or uncertain function contains an empty vulnerabilities list. The uncertain verdict is reserved for cases in which unavailable context is genuinely required to determine the security outcome.

Table 6 - Principal structured reasoning fields.

Output field	Purpose
verdict	Function-level classification: vulnerable, not_vulnerable, or uncertain.

is_vulnerable	Boolean or null representation of the function-level decision.
reason	Summarizes the overall function-level conclusion.
vulnerabilities[]	Contains zero, one, or multiple distinct vulnerability findings.
cwe / cwe_name	Identifies the primary CWE for each distinct weakness.
vulnerable_lines	Reports the source lines associated with the specific finding.
vulnerable_code_excerpt	Preserves the relevant unsafe code excerpt.
data_flow	LLM-inferred source, propagation, sink, and missing-protection explanation.
evidence_source_indices	Links a finding to supporting retrieved records where applicable.
recommended_fix	Provides remediation guidance for the identified mechanism.

4.6.3 Reasoning and Security Classification

GPT-5.1 evaluates the visible security mechanism rather than merely matching a retrieved CWE. The reasoning process considers the origin of potentially untrusted input, its propagation through the function, the sensitive operation or sink, and the presence or absence of protections such as validation, parameterization, encoding, authorization, or path containment. Existing protections are not rejected simply because a stronger alternative could be used.

The data_flow object is an LLM-generated explanatory structure and is not the output of a formal static taint-analysis engine. It is therefore used to communicate the model's interpretation of the relevant source-to-sink relationship rather than as formally verified program analysis.

4.6.4 CWE Identification and Remediation

For each confirmed vulnerability, GPT-5.1 predicts one primary CWE and reports the source lines, vulnerable code excerpt, explanation, supporting evidence identifiers, and remediation recommendation. Because the output supports multiple independent vulnerabilities per function, each finding retains its own CWE and source-location information. Severity, CVE attribution, and model-assessed confidence are not presented as final finding fields because these values cannot be inferred reliably from retrieval similarity or from a CWE label alone.

4.7 Security Report Generation

The report-generation stage is implemented in report_generation_cli.py. It reads the structured reasoning findings, summary information, and error records and produces security_report.html and security_report.json. Internal retrieval candidates, vector similarity values, source indices, and raw knowledge-base records are omitted from the user-facing report because they represent supporting context rather than confirmed evidence about the submitted project.

4.7.1 User-Facing Report Structure

The HTML report begins with project metadata and summary counts. It distinguishes the total number of analysed functions, the number of vulnerable functions, the number of distinct vulnerability findings, uncertain functions, and functions classified as not vulnerable. This distinction is necessary because a single function can contain more than one independent vulnerability.

Each confirmed vulnerability is rendered as a separate detailed finding containing the function name, file location, CWE, affected lines, technical explanation, code excerpt, and remediation guidance. Uncertain functions are presented separately with the reason for uncertainty, while non-vulnerable functions are summarized in a compact table.

4.8 End-to-End CLI Execution

The final UniXcoder-based implementation is exposed through rag_auditor.py, which coordinates preprocessing, retrieval, vulnerability reasoning, and report generation through a single command. Knowledge-base ingestion is performed separately when the persistent UniXcoder collection must be created or rebuilt.

➔ **python rag_auditor.py scan "C:\path\to\repository" --open-report**

The optional --open-report argument opens the generated HTML report after successful completion. The default retrieval depth is three records. The CLI creates a project-specific directory under processed/ and stores the extracted data, retrieval output, reasoning records, errors, summary, and final reports in that directory.

4.8.1 End-to-End Demonstration on a Vulnerable Flask Application

To demonstrate the operation of the completed prototype on a directory-based software project, the UniXcoder CLI auditor was applied to the Damn Vulnerable Flask Application repository (kolega-ai-dev/realvuln-Damn-Vulnerable-Flask-Application). This execution is presented as an end-to-end functional case study rather than as a quantitative benchmark. The purpose is to demonstrate how the implemented tool accepts a repository path, processes the contained source code, retrieves supporting knowledge, performs multi-vulnerability reasoning, and generates the final report.

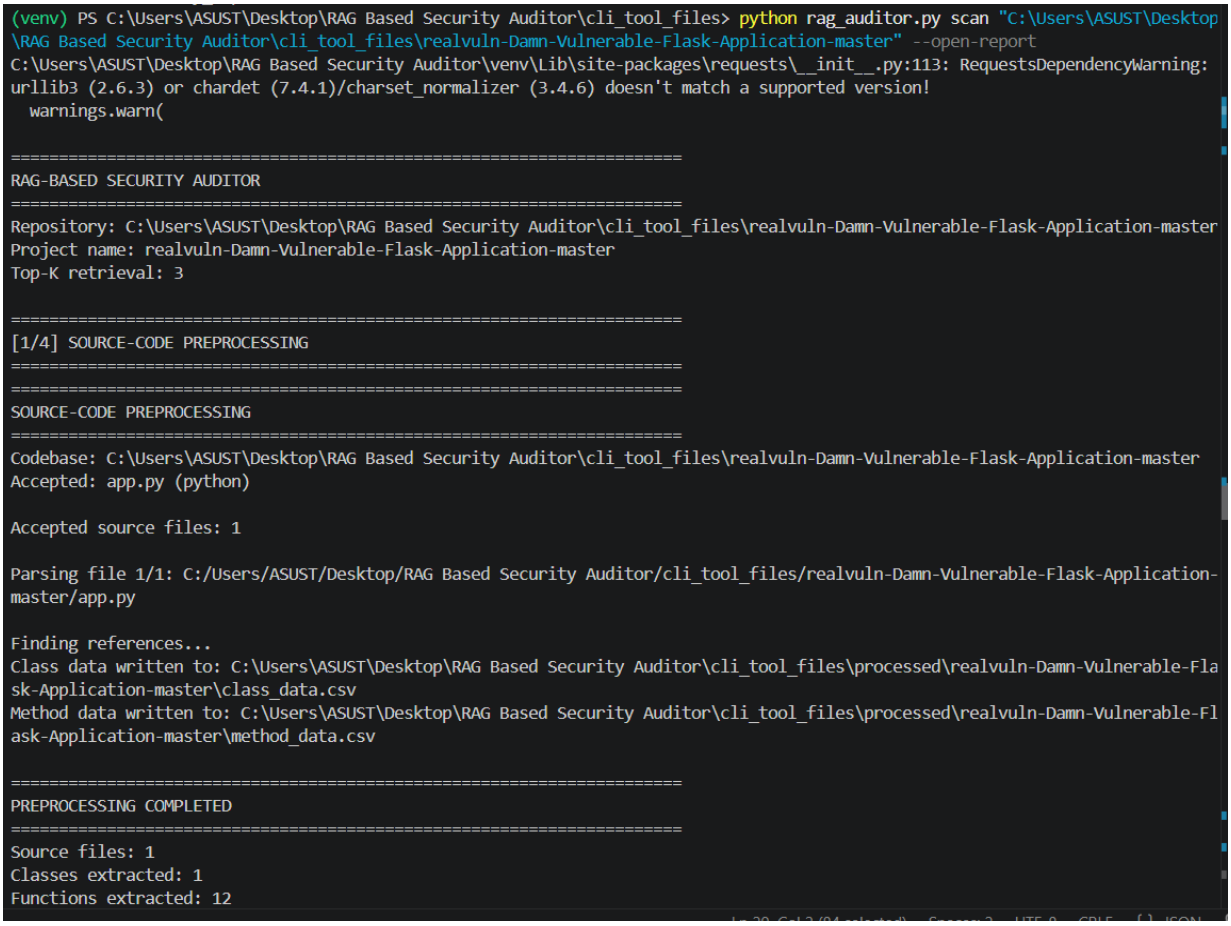

Figure 8 - Command-line execution of the auditor on the Damn Vulnerable Flask Application repository.

As shown in Figure 8, the user supplies the repository path through the scan command. The preprocessing stage accepts app.py as a supported Python source file, extracts one class and twelve functions, and writes the structural outputs to the corresponding project directory before retrieval and reasoning continue automatically.

4.8.2 Generated Security Audit Report

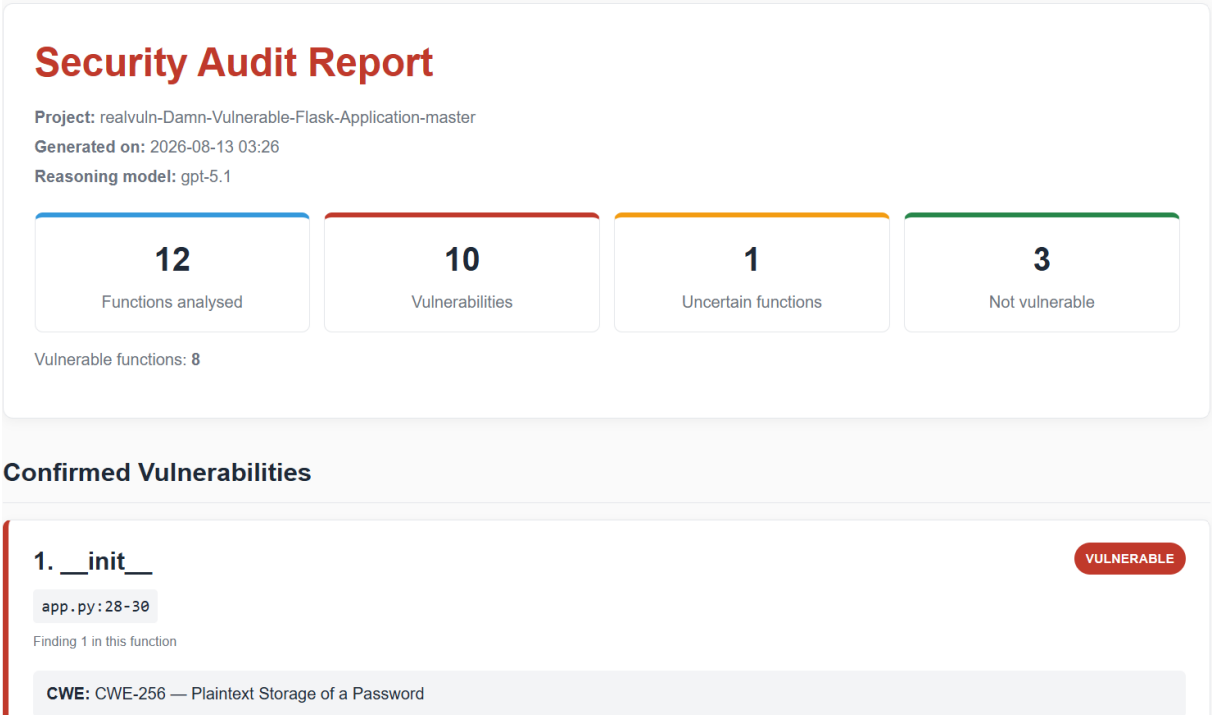

Figure 9 - Generated HTML security audit report for the Damn Vulnerable Flask Application case study.

The generated report shown in Figure 9 summarizes twelve analyzed functions. Eight functions were classified as vulnerable, with ten distinct vulnerability findings reported across them; one function was classified as uncertain and three functions were classified as not vulnerable. The difference between vulnerable functions and total vulnerabilities demonstrates the multi-vulnerability output structure, in which more than one independent weakness can be reported within the same function.

These case-study counts are used only to illustrate the behavior and presentation of the completed tool. They are not treated as benchmark true-positive, false-positive, false-negative, or true-negative results. Quantitative performance is evaluated separately using the labelled OWASP BenchmarkPython dataset in the evaluation chapter.

4.9 Implementation Boundaries

The prototype performs directory-level discovery and function-level security reasoning. Tree-sitter preserves structural boundaries and source locations, but the current system does not implement a formal whole-program taint-analysis engine, control-flow graph construction, or automated interprocedural data-flow verification. When unavailable external context is genuinely necessary to determine a function's security behavior, the reasoning schema provides an uncertain verdict rather than forcing a binary conclusion.

The two RAG configurations also retrieve different forms of evidence. BGE-M3 emphasizes functional-semantic similarity, whereas UniXcoder emphasizes direct source-code similarity. In both configurations, nearest-neighbor retrieval may return partially relevant records; therefore, retrieval is deliberately separated from the final vulnerability judgement. GPT-5.1 must verify the concrete mechanism in the submitted function before producing a confirmed finding.

4.10 Conclusion

This chapter presented the practical implementation of the RAG-based security auditor. The system processes user-specified source-code directories with Tree-sitter, uses a prepared 764-record vulnerability knowledge base, supports BGE-M3 functional-semantic retrieval and UniXcoder code-oriented retrieval, hydrates the top three retrieved records, and uses GPT-5.1 for structured function-level and multi-vulnerability reasoning.

The final command-line implementation integrates preprocessing, UniXcoder retrieval, reasoning, and reporting through rag_auditor.py and produces both HTML and JSON security audit outputs. The Damn Vulnerable Flask Application case study demonstrates the completed end-to-end workflow. The following chapter evaluates the two RAG retrieval configurations and the Bandit baseline under a common labelled benchmark.

Chapter 5: Evaluation and Results

5.1 Introduction

This chapter presents the empirical evaluation of the proposed RAG-based security auditor. The final evaluation focuses on the effect of retrieval representation on vulnerability detection. Two RAG configurations are compared: functional-semantic retrieval using BGE-M3 and direct source-code retrieval using UniXcoder. Bandit is included as a conventional static-analysis baseline. All three approaches are evaluated against the same OWASP BenchmarkPython v0.1 ground truth [22].

The central objective is not merely to maximize the number of detected vulnerabilities, but to obtain a useful balance between vulnerability coverage and false-positive control. This objective is particularly important for a practical auditing tool because excessive false alarms increase review effort and reduce developer trust in automated findings. The analysis therefore emphasizes precision, recall, F1-score, and accuracy, together with category-level behavior and the underlying TP, FP, FN, and TN counts.

5.2 Evaluation Questions

The evaluation addresses the following research questions:

- Q1: How effectively do the two RAG retrieval configurations distinguish vulnerable from non-vulnerable benchmark functions?
- Q2: How does changing the retrieval representation from BGE-M3 functional semantics to UniXcoder source-code embeddings affect detection quality and false-positive behavior?
- Q3: How do the two RAG configurations compare with Bandit, a conventional rule- and AST-based Python security analyzer?
- Q4: How does performance vary across the vulnerability categories represented in OWASP BenchmarkPython?

These questions allow the evaluation to distinguish overall detection performance from category-specific behavior and to determine whether the retrieval representation contributes materially to the usefulness of the final auditor.

5.3 Evaluation Benchmark and Experimental Scope

5.3.1 OWASP BenchmarkPython v0.1

OWASP BenchmarkPython v0.1 was selected as the primary quantitative benchmark because it provides a labelled and reproducible collection of Python security test cases with documented expected results [22]. The benchmark contains 1,230 test cases: 452 are labelled vulnerable and 778 are labelled non-vulnerable. Each test case is associated with a vulnerability category and an expected CWE, enabling controlled comparison under a common ground truth.

Table 8 - OWASP BenchmarkPython evaluation dataset.

Dataset characteristic	Value
Benchmark	OWASP BenchmarkPython v0.1
Total test cases	1,230
Vulnerable test cases	452
Non-vulnerable test cases	778
Programming language	Python
Ground-truth file	expectedresults-0.1.csv
Evaluated categories	14

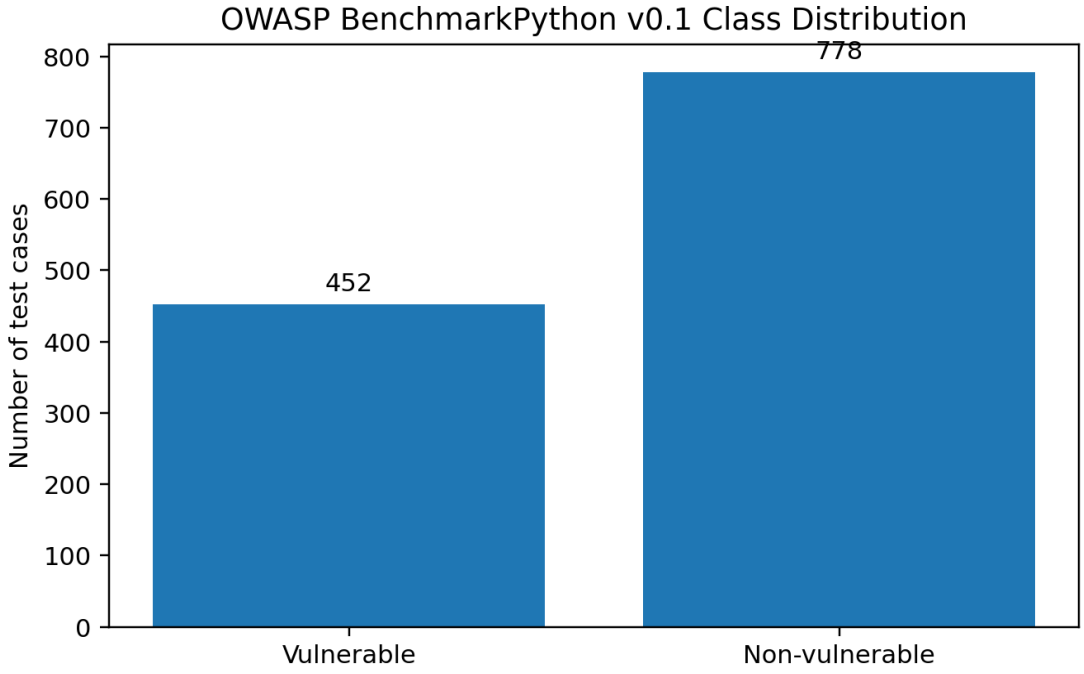

Figure 10 - Class distribution of OWASP BenchmarkPython v0.1.

5.3.2 Analysis Unit and Ground-Truth Separation

The benchmark source files were processed through the same Tree-sitter function-extraction procedure used by the auditor. One function-level prediction was retained for each benchmark identifier, producing 1,230 evaluated units. Expected vulnerability labels and expected CWE values were not supplied to the retrieval or reasoning stages. Ground truth was accessed only after prediction generation by the evaluation script.

For binary scoring, a vulnerable prediction on a vulnerable case is a true positive (TP), a vulnerable prediction on a non-vulnerable case is a false positive (FP), a non-vulnerable prediction on a vulnerable case is a false negative (FN), and a non-vulnerable prediction on a non-vulnerable case

is a true negative (TN). The system also supports an uncertain verdict. For the primary binary benchmark calculation, uncertain results are treated as non-vulnerable because they do not constitute a confirmed vulnerability report; the number of uncertain outputs is reported separately to preserve transparency.

5.3.3 Knowledge-Base Scope and Out-of-Scope CWEs

The retrieval knowledge base contains 764 prepared security records, but its coverage is not uniform across all CWE types. In this evaluation, an out-of-scope CWE refers to a benchmark weakness for which the knowledge base does not contain directly corresponding vulnerability knowledge. This differs from a sparsely represented CWE, where the identifier is present but only a limited number or limited variety of examples are available.

Out-of-scope CWEs are important when interpreting RAG results because retrieval cannot provide direct same-weakness evidence for those cases. The reasoning model can still analyze the submitted source code and may use related records, so an out-of-scope CWE does not make detection impossible. However, the absence of directly relevant examples can reduce the usefulness of retrieval, increase dependence on the reasoning model's pretrained knowledge, and contribute to missed detections or less precise evidence selection. For this reason, the main results are reported over all 1,230 benchmark cases rather than filtering the benchmark to only knowledge-base-supported weaknesses. This provides a more conservative assessment of the complete auditing pipeline.

5.4 Evaluated Systems

5.4.1 UniXcoder RAG Configuration

The UniXcoder configuration performs code-oriented retrieval. The vulnerable_code field of each knowledge-base record is embedded using microsoft/unixcoder-base, while each benchmark function is embedded directly from its original source code. The three nearest records are retrieved from ChromaDB and hydrated from final-knowledge-base.jsonl before being provided to GPT-5.1. No GPT-5-mini functional-description call is required in this configuration.

5.4.2 BGE-M3 RAG Configuration

The BGE-M3 configuration performs functional-semantic retrieval. GPT-5-mini first generates a concise functional description of each benchmark function. BAAI/bge-m3 embeds that description and retrieves the three nearest knowledge-base records based on stored functional_description embeddings. GPT-5.1 then receives the original function, its description, and the retrieved records for the final security assessment.

5.4.3 Bandit Static-Analysis Baseline

Bandit was selected as the conventional static-analysis baseline because it is a Python security linter that parses source code into an abstract syntax tree and applies a collection of security-

oriented plugins to relevant syntax nodes [35]. Unlike the RAG configurations, Bandit does not use an external vulnerability knowledge base or generative reasoning. Its detections are determined by predefined checks for patterns such as unsafe function calls, weak cryptographic usage, shell execution, insecure temporary-file handling, and other Python security concerns.

Bandit version 1.7.9 was executed recursively over the benchmark source directory using its default rule set, without manual rule selection or tuning after observing the benchmark outcomes. For fair binary detection scoring, a benchmark function was considered predicted vulnerable when Bandit reported at least one finding within that function's source-line range. This evaluation therefore measures whether the tool raised a security finding for the target function rather than requiring Bandit's rule identifier to match the benchmark CWE before counting a detection.

Table 9 - Experimental configuration.

Component	UniXcoder RAG	BGE-M3 RAG	Bandit
Analysis unit	Tree-sitter function	Tree-sitter function	Function source-line range
Retrieval representation	Raw source code	Functional description	Not applicable
Embedding model	microsoft/unixcoder-base	BAAI/bge-m3	Not applicable
Description model	Not required	GPT-5-mini	Not applicable
Knowledge base	764 records	764 records	Not applicable
Retrieved evidence	Top 3	Top 3	Not applicable
Reasoning model	GPT-5.1	GPT-5.1	Rule/plugin based
Verdicts	vulnerable / not_vulnerable / uncertain	vulnerable / not_vulnerable / uncertain	finding / no finding

5.5 Evaluation Procedure and Metrics

The evaluator maps every generated prediction to its corresponding BenchmarkTest identifier and compares the result with the official expected-results file. Four metrics are used in the main comparison: precision, recall, F1-score, and accuracy. These measures were selected because together they describe the reliability of positive findings, vulnerability coverage, the balance between those two objectives, and overall classification correctness.

5.5.1 Precision

Precision measures the proportion of reported vulnerabilities that are actually labelled vulnerable by the benchmark. It is particularly important for this project because higher precision means fewer false alarms among the findings that developers must inspect.

➔ **Precision = TP / (TP + FP)**

5.5.2 Recall

Recall measures the proportion of all benchmark vulnerabilities that are successfully detected. In security auditing, recall represents vulnerability coverage: a low-recall detector can appear precise while still overlooking many genuine weaknesses.

➔ **Recall = TP / (TP + FN)**

5.5.3 F1-score

The F1-score is the harmonic mean of precision and recall. It is useful when neither false positives nor missed vulnerabilities should dominate the evaluation, and it provides a single summary of the trade-off between detection coverage and positive-prediction reliability.

➔ **F1 = 2 × (Precision × Recall) / (Precision + Recall)**

5.5.4 Accuracy

Accuracy measures the proportion of all vulnerable and non-vulnerable benchmark cases classified correctly. It provides a broad view of overall correctness, but it is interpreted together with precision and recall because the benchmark contains more non-vulnerable than vulnerable cases.

➔ **Accuracy = (TP + TN) / (TP + FP + FN + TN)**

5.6 Overall Results

5.6.1 Confusion-Matrix Results

Table 10 - Confusion-matrix outcomes and uncertain predictions.

System	TP	FP	FN	TN	Uncertain
UniXcoder RAG	413	188	39	590	19
BGE-M3 RAG	413	263	39	515	13
Bandit	224	128	228	650	0

Both RAG configurations detected 413 of the 452 vulnerable benchmark cases, leaving 39 false negatives and therefore producing identical overall recall. Their behavior on non-vulnerable code was substantially different. UniXcoder produced 188 false positives, whereas BGE-M3 produced 263. Consequently, direct code-oriented retrieval removed 75 false-positive predictions without reducing the total number of detected vulnerabilities.

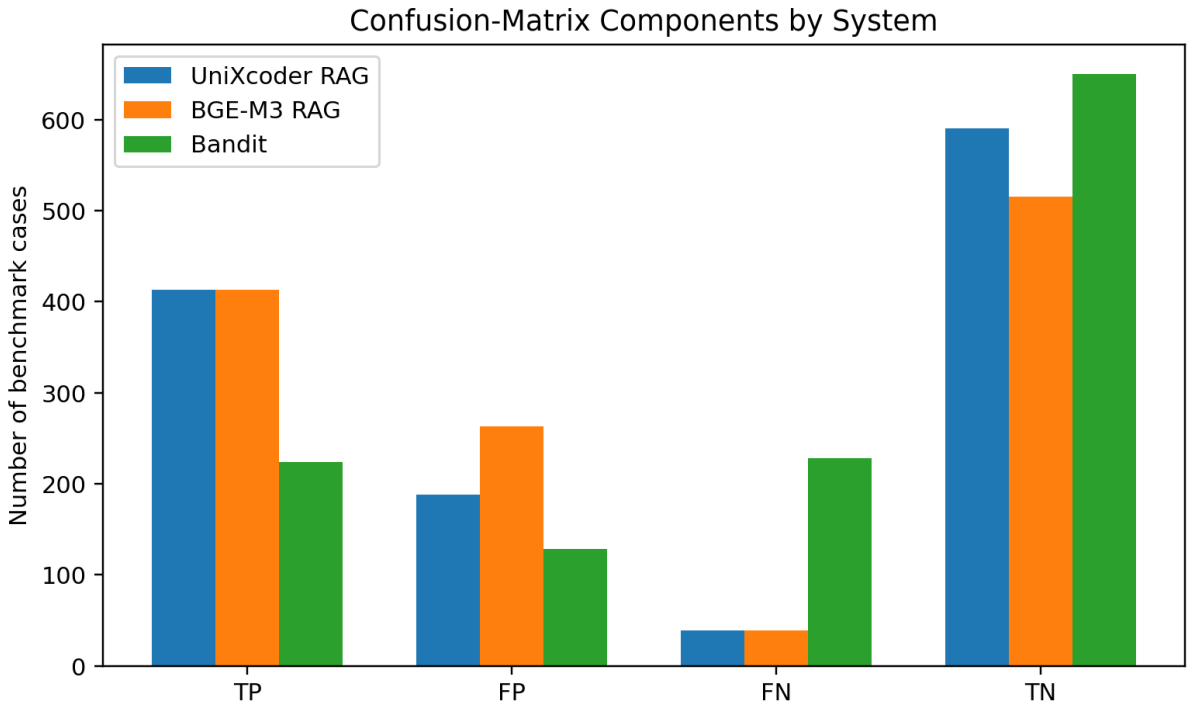

Figure 11 - Confusion-matrix components for the evaluated systems.

5.6.2 Detection Metrics

Table 11 - Overall detection performance.

System	Precision	Recall	F1-score	Accuracy
UniXcoder RAG	68.72%	91.37%	78.44%	81.54%
BGE-M3 RAG	61.09%	91.37%	73.23%	75.45%
Bandit	63.64%	49.56%	55.72%	71.06%

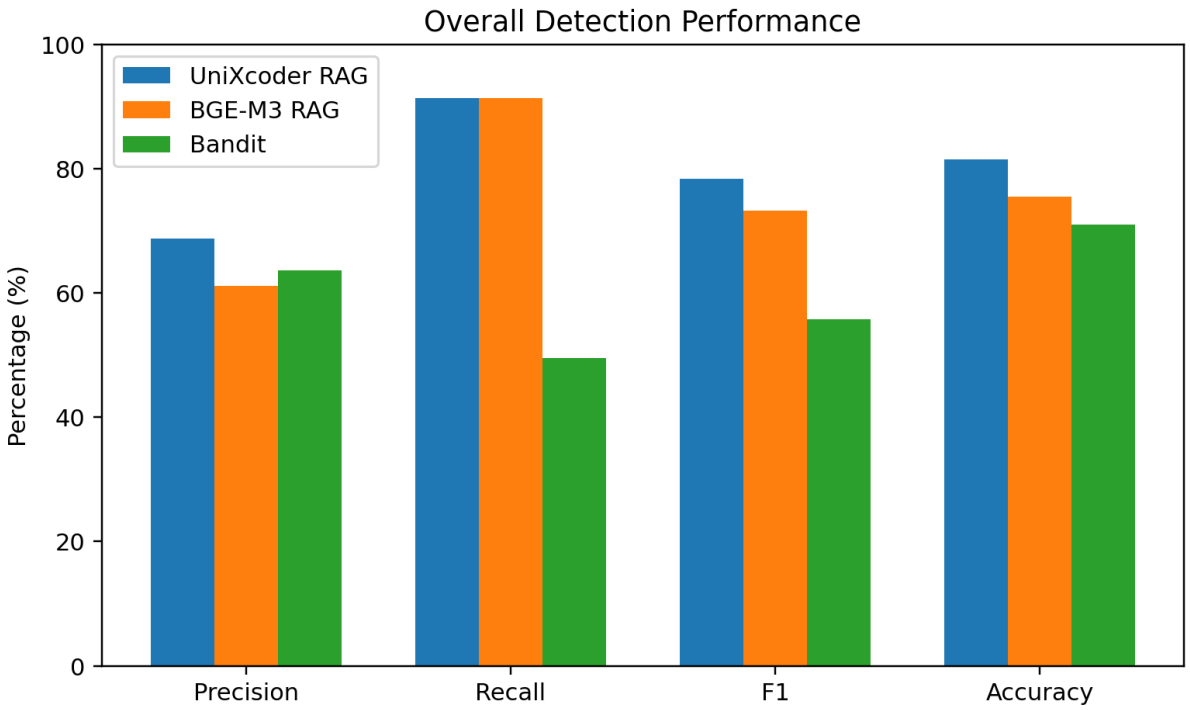

Figure 12 - Overall precision, recall, F1-score, and accuracy.

UniXcoder achieved the strongest overall balance, with 68.72% precision, 91.37% recall, a 78.44% F1-score, and 81.54% accuracy. BGE-M3 achieved the same 91.37% recall but lower precision (61.09%), F1-score (73.23%), and accuracy (75.45%). The identical recall is important because the UniXcoder improvement was not obtained by simply becoming more conservative and rejecting more vulnerable cases. Instead, the main improvement came from more accurate rejection of benchmark-safe code.

Relative to BGE-M3, UniXcoder increased precision by 7.63 percentage points, F1-score by 5.21 points, and accuracy by 6.09 points while preserving the same vulnerability coverage. This result directly supports the project's emphasis on false-positive control: changing the retrieval representation altered the quality of evidence supplied to the same downstream reasoning stage and materially improved the usefulness of the resulting findings.

Bandit achieved 63.64% precision, 49.56% recall, a 55.72% F1-score, and 71.06% accuracy. Its lower recall reflects the coverage boundaries of predefined security rules. UniXcoder detected 189 more true vulnerabilities than Bandit (413 versus 224) while also achieving higher precision, F1-score, and accuracy.

5.7 Category-Level Results

Table 12 - Recall and F1-score by vulnerability category (%).

Category	Uni Recall	Uni F1	BGE Recall	BGE F1	Bandit Recall	Bandit F1
cmdi	92.3	88.9	100.0	92.9	100.0	78.8
codeinj	100.0	76.9	100.0	75.5	100.0	54.8
deserialization	100.0	78.3	100.0	76.6	100.0	75.0
hash	100.0	92.8	98.6	68.3	100.0	100.0
ldapi	100.0	80.0	100.0	78.0	0.0	0.0
pathtraver	100.0	70.7	100.0	71.4	0.0	0.0
redirect	100.0	86.7	100.0	89.7	0.0	0.0
securecookie	100.0	100.0	100.0	100.0	0.0	0.0
sqli	100.0	100.0	100.0	100.0	100.0	47.6
trustbound	11.1	20.0	11.1	20.0	0.0	0.0
weakrand	81.8	88.5	87.9	84.5	77.8	87.5
xpathi	98.0	65.8	96.1	62.8	23.5	22.2
xss	90.3	70.0	74.2	61.3	0.0	0.0
xxe	100.0	57.1	100.0	57.1	100.0	44.4

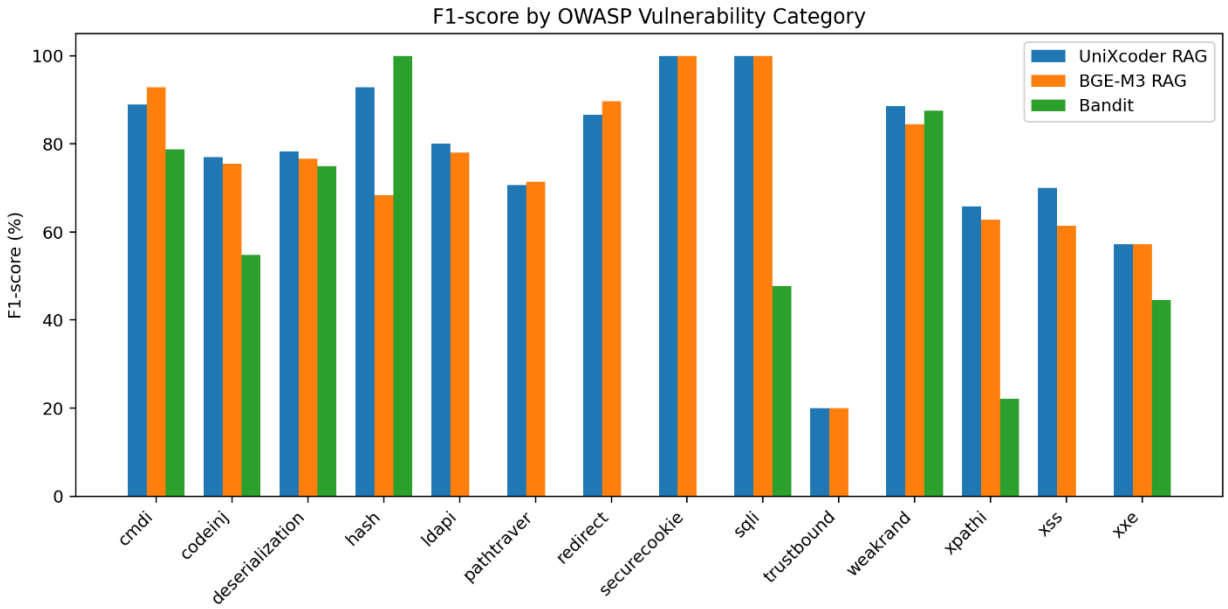

Figure 12 - F1-score comparison across vulnerability categories.

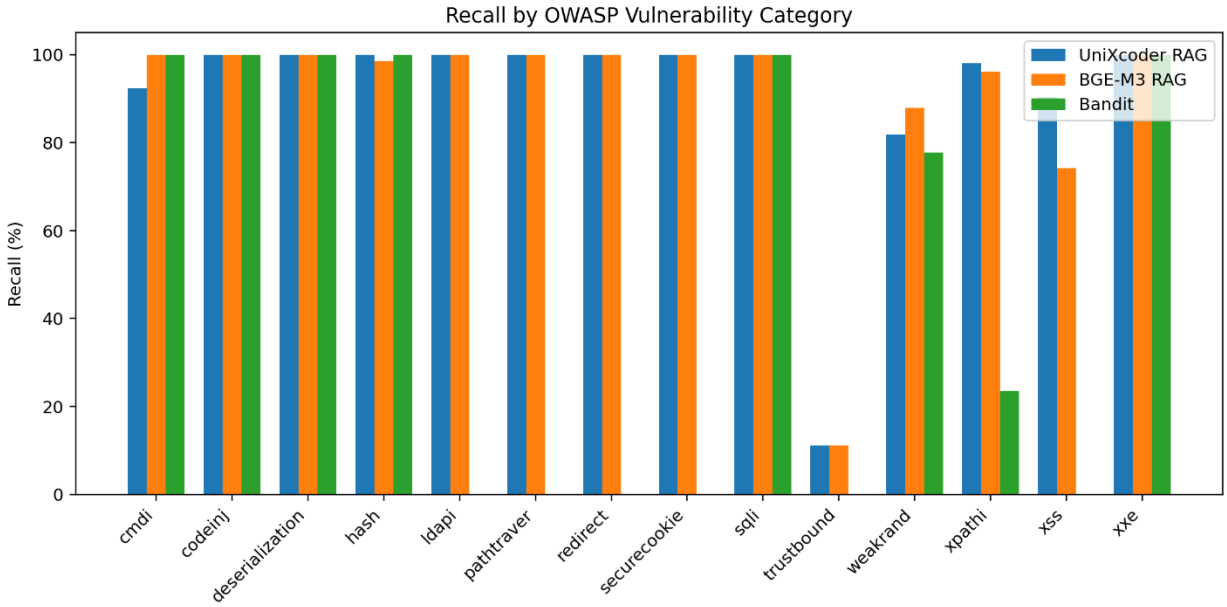

Figure 14 - Recall comparison across vulnerability categories.

5.7.1 Comparison of the Two RAG Representations

The category-level results show that the two RAG configurations are not uniformly interchangeable. UniXcoder obtained a higher F1-score in seven of the fourteen categories, BGE-M3 was higher in three, and four categories were tied. Both configurations achieved perfect F1-score on SQL injection and secure-cookie cases, while both remained weak on the trust-boundary category.

The largest practical improvement occurred in the hash category. BGE-M3 produced 64 false positives in this category, whereas UniXcoder produced only 11, while UniXcoder still detected all 71 vulnerable cases. This changed category F1 from 68.29% to 92.81%. Weak-randomness also showed a substantial reduction in false positives, from 20 under BGE-M3 to 3 under UniXcoder, although BGE-M3 retained somewhat higher recall in that category. These results indicate that direct code representation can preserve implementation details that help the reasoning model distinguish insecure operations from superficially similar but safe implementations.

UniXcoder also improved XSS recall from 74.19% to 90.32% and F1-score from 61.33% to 70.00%, while improving F1 in code injection, deserialization, LDAP injection, XPath injection, and weak randomness. BGE-M3 remained slightly stronger for command injection, path traversal, and redirect cases. This suggests that functional-semantic abstraction remains useful for some vulnerability mechanisms, particularly when behavioral similarity is more informative than implementation-level code structure.

5.7.2 Comparison with Bandit by Category

Bandit exhibited a much more category-dependent profile. It achieved perfect binary detection performance in the hash category and strong performance for weak randomness, command

injection, and deserialization, which correspond well to patterns covered by explicit security rules. In contrast, it produced no true-positive detections for LDAP injection, path traversal, redirect, secure-cookie, trust-boundary, or XSS benchmark cases under the default configuration.

The RAG configurations therefore provide broader vulnerability-category coverage than the default Bandit rule set. This does not imply that generative reasoning should replace static analysis. Rather, the results show complementary strengths: rule-based analysis can be highly effective when a dedicated check matches the relevant pattern, while retrieval-grounded reasoning can recognize a wider range of security mechanisms that are not fully captured by the default rule inventory.

5.8 Discussion of Improvements and Practical Usefulness

The most important finding is that retrieval design affected practical detection quality even though both RAG configurations used the same 764-record knowledge source, the same top-three retrieval depth, and GPT-5.1 as the downstream reasoning model. BGE-M3 abstracts a function into a concise description of what it does. This can retrieve behaviorally similar examples across different implementations, but the abstraction can omit details such as safe APIs, parameterization, escaping, validation, or exact call structure. UniXcoder instead retrieves from the submitted source code itself, preserving more of these implementation-level distinctions.

The overall results are consistent with this design difference. UniXcoder preserved the 91.37% recall achieved by BGE-M3 while eliminating 75 false positives. This is particularly relevant to the intended use of the project as a developer-facing auditing tool. A high-recall system is useful for discovering candidate vulnerabilities, but practical adoption also depends on limiting unnecessary findings. The increase in precision from 61.09% to 68.72% means that a larger proportion of the issues presented to a developer correspond to benchmark-labelled vulnerabilities.

The comparison with Bandit further shows the value of using the system as a complementary security-analysis layer. Bandit is inexpensive, deterministic, and effective for rule-covered patterns, whereas the RAG auditor provides substantially broader recall and can generate structured explanations, affected lines, CWE-oriented findings, and remediation guidance. A practical workflow can therefore use conventional static analysis for fast rule-based checks and the RAG auditor for additional function-level review where broader contextual reasoning is beneficial.

The results should not be interpreted as evidence that RAG is automatically superior independent of retrieval design. The difference between BGE-M3 and UniXcoder demonstrates the opposite: retrieval representation is a significant design choice. The external knowledge is useful only when the retrieval mechanism supplies sufficiently relevant evidence and the reasoning model correctly verifies the vulnerability mechanism in the target code.

5.9 Computational Cost and API Usage

The two RAG configurations differ in API usage. UniXcoder performs source-code embedding locally and therefore requires one GPT-5.1 reasoning call per benchmark function. BGE-M3 adds a GPT-5-mini description-generation call before the common GPT-5.1 reasoning stage. The embedding models and ChromaDB execute locally and do not incur per-token API charges.

Table 13 - Approximate API usage and monetary cost for the 1,230-case benchmark.

Cost item	UniXcoder RAG	BGE-M3 RAG	Bandit
GPT-5.1 calls	1,230	1,230	0
GPT-5-mini calls	0	1,230	0
Approx. GPT-5.1 input tokens	~2.10 M	~2.10 M	N/A
Approx. GPT-5.1 output tokens	~0.18 M	~0.18 M	N/A
Approx. GPT-5-mini input tokens	N/A	~0.12 M	N/A
Approx. GPT-5-mini output tokens	N/A	~0.075 M	N/A
Approx. API cost	~$4.43	~$4.61	$0 API cost

The estimates use the stated standard GPT-5.1 text pricing of $1.25 per million input tokens and $10.00 per million output tokens, together with standard GPT-5-mini text pricing of $0.25 per million input tokens and $2.00 per million output tokens. Under the approximate token volumes shown above, the UniXcoder benchmark run costs about $4.43 in API usage, while the BGE-M3 configuration costs about $4.61. The difference is modest in monetary terms for this benchmark, but UniXcoder removes 1,230 description-generation API calls and therefore also reduces pipeline latency and operational complexity. Actual billing varies with exact prompt length, response length, cached tokens, and API settings.

5.10 Threats to Validity and Evaluation Limitations

5.10.1 Benchmark Validity

OWASP BenchmarkPython provides controlled labels and reproducible scoring, but purpose-built benchmark cases do not represent every characteristic of production repositories. A finding that is plausible under a broader deployment assumption is still counted according to the official benchmark label.

5.10.2 Language Scope

The quantitative evaluation is restricted to Python. The implementation supports Python, JavaScript, TypeScript, and TSX preprocessing, but equivalent labelled evaluation corpora were not used for the other supported languages. The reported performance therefore establishes effectiveness only for the evaluated Python benchmark setting.

5.10.3 Knowledge-Base Coverage

Knowledge-base coverage varies across CWE types. Out-of-scope or sparsely represented weaknesses may receive less useful retrieval evidence, which can affect both missed detections and false-positive behavior. Because the complete benchmark was retained rather than filtering unsupported cases, the reported results include this limitation.

5.10.4 Function-Level Reasoning

The system analyzes individual functions rather than performing formal whole-program or interprocedural taint analysis. Vulnerabilities that depend on caller behavior, templates, global configuration, shared state, or code in another file can therefore remain ambiguous. The uncertain verdict provides a way to represent such cases without treating every incomplete context as either definitely vulnerable or definitely safe.

5.10.5 Model Variability

GPT-5.1 is a generative model and repeated executions can vary. The results represent complete benchmark runs under the evaluated configurations. The comparison should therefore be interpreted as empirical performance under these fixed runs rather than as a deterministic guarantee for every future execution.

5.11 Future work

- Cross-Language Evaluation:
  The current quantitative evaluation was conducted using OWASP BenchmarkPython v0.1. Although the implemented preprocessing pipeline supports Python, JavaScript, TypeScript, and TSX, equivalent labelled benchmarks were not available for evaluating the other supported languages under the same experimental conditions. Future work should therefore extend the evaluation to JavaScript, TypeScript, and TSX when sufficiently comprehensive vulnerability benchmarks become available.

- Expansion of the Vulnerability Knowledge Base:
  The current knowledge base contains 764 security records, but its coverage is not uniform across all CWE categories. Some weaknesses are represented by several examples, while others are sparsely represented or remain outside the knowledge-base scope. Future work should expand the knowledge base with additional vulnerable and secure implementations covering a broader range of CWEs, programming languages, frameworks, and coding

patterns. Increasing the diversity of secure examples may also help improve false-positive control.

- Improved Coverage of Out-of-Scope and Sparsely Represented CWEs:
  The evaluation showed that the effectiveness of retrieval can depend on the availability and diversity of relevant knowledge records. Future extensions should specifically target CWE categories that are currently absent or poorly represented. This would allow the retrieval stage to provide more directly relevant evidence and reduce dependence on the reasoning model's pretrained knowledge when analyzing less represented vulnerability types.

- Repository-Level Contextual Analysis:
  The current system performs directory-level source-code discovery but primarily conducts vulnerability reasoning at the function level. Future work could incorporate additional repository context, including callers, callees, imported functions, configuration files, templates, class relationships, and relevant cross-file references. This would improve the analysis of vulnerabilities whose security implications depend on information located outside an individual function.

- Program-Analysis Support:
  Future versions could complement the existing Tree-sitter preprocessing stage with program-analysis information such as call graphs, control-flow information, or lightweight data-flow tracking. These additional representations could provide more explicit information about how potentially untrusted data propagates across functions and components, particularly for vulnerabilities that cannot be determined reliably from source-code inspection alone.

- Retrieval-Quality Improvement:

  The current system retrieves the top three knowledge-base records for each analysed function. Future work could investigate improved candidate-selection techniques, such as retrieval relevance thresholds, reranking, or diversity-aware selection. These mechanisms could reduce the influence of weakly related retrieved examples and provide the reasoning model with more focused supporting evidence.

- Further False-Positive Reduction:
  The evaluation demonstrated that UniXcoder reduced false positives compared with the BGE-M3 configuration while maintaining the same overall recall. Nevertheless, false positives remain an important consideration for practical security auditing. Future research should perform detailed category-level analysis of recurring false-positive patterns and

improve the system's ability to recognize effective protections such as validation, parameterization, output encoding, authorization checks, and secure API usage.

- Performance and Cost Optimization:
  The current pipeline requires GPT-5.1 reasoning for each analyzed function, while the BGE-M3 configuration additionally requires GPT-5-mini functional-description generation. Future work could investigate batching, caching repeated analyses, selective reasoning of higher-risk functions, more efficient prompt construction, and suitable local models to reduce execution time and API cost while preserving detection quality.

5.12 Conclusion

This chapter evaluated UniXcoder RAG, BGE-M3 RAG, and Bandit on all 1,230 OWASP BenchmarkPython v0.1 test cases. UniXcoder achieved the strongest overall result with 68.72% precision, 91.37% recall, 78.44% F1-score, and 81.54% accuracy. BGE-M3 achieved the same recall but lower precision, F1-score, and accuracy because it produced 75 additional false positives. Bandit achieved substantially lower recall and F1-score, although it remained strong in categories closely aligned with dedicated default rules.

The results show that the main enhancement of the final system is not simply the addition of retrieval, but the use of a retrieval representation that better preserves security-relevant implementation details. UniXcoder retained broad vulnerability coverage while improving false-positive control, making the resulting audit output more useful for practical developer review. At the same time, category-level differences show that semantic and code-oriented retrieval have complementary strengths and that RAG performance remains dependent on knowledge-base coverage and the quality of the retrieved evidence.

References

[1] "About CWE," MITRE Corporation, 2024.

[2] S. Chakraborty, R. Krishna, Y. Ding and B. Ray, "Deep Learning based Vulnerability Detection: Are We There Yet?," arXiv, 2020.

[3] V. B. Zoltán Szabó, "A New Approach to Web Application Security: Utilizing GPT Language Models for Source Code Inspection," *Future Internet,* 2023.

[4] T. Z. D. L. Xin Zhou, "Large Language Model for Vulnerability Detection: Emerging Results and Future Directions," in *ICSE-NIER'24 — Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 2024.

[5] G. Z. K. W. Y. Z. Y. W. W. D. J. F. M. L. B. C. X. P. T. M. Y. L. XUEYING DU, "Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG," *ACM Transactions on Software Engineering and Methodology,* p. 26, 2026.

[6] S. X. P. Z. K. L. D. L. Z. L. Jianlv Chen, "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv (Computer Science → Computation and Language), 2025.

[7] S. L. N. D. Y. W. M. Z. J. Y. Daya Guo, "UniXcoder: Unified Cross-Modal Pre-training for Code Representation," *arXiv,* 2022.

[8] T. P. Contributors, "Tree-sitter Official Documentation," [Online]. Available: https://tree-sitter.github.io/tree-sitter .

[9] N. S. N. P. J. U. L. J. A. N. G. Ł. K. I. P. Ashish Vaswani, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 2017.

[10] M. C. e. al., "Evaluating Large Language Models Trained on Code," arXiv (Machine Learning — cs.LG), 2021.

[11] Y. L. L. C. A. D. C. L. L. T. F. X. H. E. Z. Y. Z. Y. C. L. W. A. T. L. W. B. F. S. S. S. Yue Zhang, "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," *Computational Linguistics (MIT Press),* 2025.

[12] E. P. A. P. F. P. V. K. N. G. H. K. M. L. W. Y. T. R. S. R. D. K. Patrick Lewis, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems 33*, 2020.

[13] H. H. F. L. Q. Y. J. W. Z. L. P. L. M. W. H. Z. Y. L. Yanming Mu, "Towards Secure Retrieval-Augmented Generation: A Comprehensive Review of Threats, Defenses and Benchmarks," *arXiv,* 2026.

[14] B. O. S. M. P. L. L. W. S. E. D. C. W. Y. Vladimir Karpukhin, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[15] "Chroma Docs," [Online]. Available: https://docs.trychroma.com/docs/collections/configure.

[16] J. K. T. A. R. C. L. A. S. &. P. T. Samuele Pasini, "Evaluating and improving the robustness of security attack detectors generated by LLMs," *Empir Software Eng,* pp. 31-35, 2026.

[17] X. C. ,. L. ,. X. C. G. Zhijie Chen, "ReVul-CoT: Towards Effective Software Vulnerability Assessment with Retrieval-Augmented Generation and Chain-of-Thought Prompting," arXiv, China, 2025.

[18] M. M. A. A. Md Hasan Saju, "An Empirical Evaluation of LLM-Based Approaches for Code Vulnerability Detection: RAG, SFT, and Dual-Agent Systems," *arXiv,* p. 6, 2026.

[19] N. R. A. R. A. S. I. G. Nandan Thakur, "BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models," arXiv (Information Retrieval — cs.IR), 2021.

[20] F. R. John Pellew, "RealVuln Benchmarking Rule-Based, General-Purpose LLM, and Security-Specialized Scanners on Real-World Code," *arXiv,* 2026.

[21] S. S. K. K. S. T. M. Subho Halder, "Seclens: Role-specific Evaluation of LLM's for security vulnerablity detection," *arXiv,* 2026.

[22] O. Foundation, "OWASP Benchmark Project," OWASP, [Online]. Available: https://owasp.org/www-project-benchmark/.

[23] S. G. T. Y. L. Y. F. C. W. D. M. D. Alperen Yildiz, "Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories," *Association for Computational Linguistics (ACL),* pp. 30848--30865, 2025.

[24] M. H. S. P. H. P. A. C. G. S. Saad Ullah, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet) A Comprehensive Evaluation, Framework, and Benchmarks," *arXiv,* 2024.

[25] L. S. J. R. G. M. B. E. H. D. B. M. B. J. C. D. Z. Nancy Lau, "ZeroDayBench Evaluating LLM Agents on Unseen Zero-Day Vulnerabilities for Cyberdefense," *arXiv,* 2026.

[26] S. K. D. K. J. S. C. Chinmay Pushkar, "Beyond Single Bugs: Benchmarking Large Language Models for Multi-Vulnerability Detection," *arXiv,* 2025.

[27] N. T. B. T. Z. C. Y. X. Z. M. W. J. J. J. C. H. H. H. H. N. C. Y. H. J. T. R. L. Y. Y. H. W. A. F. L. E. L. O. L. K. S. D. L. Yikun Li, "Out of Distribution, Out of Luck: How Well Can LLMs Trained on Vulnerability Datasets Detect Top 25 CWE Weaknesses?," *arXiv,* 2026.

[28] C. a. I. E. a. S. R. Tony, "Retrieve, Refine, or Both? Using Task-Specific Guidelines for Secure Python Code Generation," *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME),* pp. 368-379, 2025.

[29] W. C. S. S. M. Y. Minjae Seo∗, "AutoPatch: Multi-Agent Framework for Patching Real-World CVEs Generated by Outdated LLMs," arXiv, 2025.

[30] S. L. Y. Z. X. L. L. Z. Fang Liu1, "SecureReviewer: Enhancing Large Language Models for Secure Code Review through Secure-aware Fine-tuning," in *48th International Conference on Software Engineering (ICSE 2026)*, Rio de Janeiro, Brazil, 2026.

[31] Y. L. X. S. Di Cao, "RealVul: Can We Detect Vulnerabilities in Web Applications with LLM?," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, Miami, Florida, USA, 2024.

[32] J. G. 1. C. 1. X. X. 1. Z. S. 1. X. Z. 1, "REPOAUDIT: An Autonomous LLM-Agent for Repository-Level Code Auditing," in *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, Vancouver, Canada, 2025.

[33] J. X. Y. P. C. Y. C. a. X. J. Zihan Wu, "MulVul: Retrieval-augmented Multi-Agent Code Vulnerability Detection via Cross-Model Prompt Evolution," arXiv, 2026.

[34] S. Thornton, "SecureCode v2.0: A Production-Grade Dataset for Training Security-Aware Code Generation Models," *arXiv,* 2025.